

Model updates: batched refresh, maintained state, and the refresh surface

Andrew Browne

2026-07-03

Contents

Why this document exists	2
Part 1 — The field: two camps, one parent contract	3
Part 2 — Batched refresh: mechanism and the rejection catalogue	5
Part 3 — Filter placement: eligibility is pushdown depth	8
Part 4 — The monotonicity primitive	12
Part 5 — Condition deep-dive: set operations at the base (E1)	19
Part 6 — Condition deep-dive: subquery in FROM (B4 / E2)	21
Part 7 — Condition deep-dive: joins (the un-gated construct)	25
Part 8 — Condition deep-dive: window functions, LAG/LEAD , and the two-layer lookback (B1 / C1)	29
Part 9 — Shorter conditions	34
Part 10 — Output-time granularity and the run-window / partition alignment	40
Part 11 — Window independence: run order and parallelism	42
Part 12 — Prior art and external validation	44
Part 13 — Maintained refresh: the two axes, the contract, and hidden state	48
Part 14 — The algebraic ladder: the maintained eligibility boundary	50
Part 15 — Emulation vs delegation	52
Part 16 — Ontology: the maintained umbrella	53
Part 17 — The user surface: explicit materialization modes	54
Part 18 — Open questions	59

Status: research (decision-oriented, living document) **Date:** 2026-07-03 **Owners:** andrew **Related:**
 - Spec: [docs/specs/incremental_models.md](#) — the window-forward (batched) mode this document audits. - Spec: [docs/specs/cumulative_aggregate.md](#) — the first shipped member of the maintained family. - Spec: [docs/specs/multi_backend.md](#) — the capability matrix Part 15 extends with `supports_native_ivm` / `supports_retraction`. - Spec: [docs/specs/models.md](#) §“Refresh axis” — where the refresh values of Part 17 live. - Plan: [docs/plans/20260702-monotonicity-primitive-tested.md](#) — implements the Part 4 primitive. - Research: [docs/research/2026-05-20-incremental-gaps-from-web-analytics.md](#) - Research: [docs/research/20260521-incremental-as-planner-rule.md](#) - Research: [docs/research/20260522-cumulative-as-its-own-rule.md](#) — why cumulative is its own rule; the sibling-rule sketches (`scd2`, `latest_value`, `accumulating_snapshot`). - Supporting material: [docs/research/20260701-monotonicity-primitive-research/](#) (deep-dive notes behind Parts 4 and 12); empirical harnesses in [docs/research/harness/](#).

This document replaces two earlier research docs ([20260701-expanding-incremental-eligibility.md](#) and [20260703-maintained-refresh-and-hidden-state.md](#)); links in older plans point at those filenames.

Why this document exists

Every stored smelt model has to answer one operational question: **how is it kept up to date as new data arrives?** This document is the shared home for that whole topic — *model updates* — covering three connected bodies of work:

1. **The batched (window-forward) audit** — **Parts 2–12.** smelt currently **rejects** a model from incremental materialization in a large number of situations. Some rejections are genuine correctness requirements; others are *conservative* — the general case is unsafe, but a well-characterised sub-case is provably fine and is exactly the pattern users reach for. Every unnecessary rejection pushes a user onto a full-table rebuild (or off smelt), so the rejections are audited one by one: why each is rejected, whether it is a correctness law or a mechanical limitation, and what a safe relaxation requires. All condition analyses converge on two shared foundations — a **filter-placement theory** (Part 3) and a **monotonicity primitive** (Part 4, now implemented) — and are validated empirically (DuckDB harnesses) and externally (Part 12).
2. **The maintained-state camp** — **Parts 13–16.** smelt’s **refresh**: **cumulative** is its one foothold in the *other* way the field keeps models current: maintained views over hidden state. Generalizing cumulative along two axes (state representation; maintainer) opens a design space that reaches from **AVG-via-(sum, count)** on DuckDB all the way to delegating maintenance to Databricks’ native incremental-view runtime — under one equivalence contract.
3. **The user surface** — **Part 17.** What a modeller actually declares: an explicit, flat enum of refresh modes (`full` | `batched` | `cumulative` | `versioned` | `latest_value` | `materialized_view`), each naming exactly one contract, with the analysis machinery of Parts 2–16 acting as *validator*, never as a silent chooser.

Part 18 collects the open questions — including which of them the combined scope settles and which it newly creates.

A note on names. Part 17 recommends renaming the window-forward mode from **incremental** to **batched**, freeing “incremental” to be the *family* word (everything here is incremental in the broad, incremental-view-maintenance sense). The shipping surface, specs, and code today still spell the mode

incremental:, so Parts 2–12 — which audit that implementation — use “incremental” throughout; read it as the mode Part 17 calls **batched**.

Part 1 — The field: two camps, one parent contract

1.1 Two ways to keep a model current

Production systems split cleanly into two camps, and the split determines almost everything downstream — what analysis is needed, what state is kept, and what the user must guarantee:

- **Window-forward over a monotone event-time** (smelt’s incremental, Part 17’s **batched**): read the next time window and assume the source is append-only/monotone so earlier windows are settled; **DELETE+INSERT** that window’s partitions. Shared by **cube.dev** (requires a `time_dimension`), **ClickHouse** MVs (append-only insert blocks), **dbt microbatch** (`event_time`), **SQLMesh** `INCREMENTAL_BY_TIME_RANGE` (`time_column`), and — in streaming form — **Spark Structured Streaming** and **Flink** (the watermark *is* the monotone-event-time assertion). Simple, needs no change-tracking metadata, but pays the monotonicity price (Part 4) and only covers the monotone/linear operator slice (§12.2).
- **Change-tracking / delta-diffing the source** (the maintained camp): detect *which rows changed* and propagate the delta into maintained state. Shared by **Snowflake Dynamic Tables** (Stream-style change tracking), **BigQuery** MVs (storage-metadata append diffing), **Databricks Enzyme** (Delta row-tracking
 - change data feed), **Materialize**, **Flink**, and — the theoretical endpoint — **Feldera/DBSP** (Z-sets: every row carries a $\pm$ weight, so inserts/updates/deletes propagate uniformly). No monotone column needed; covers joins, **DISTINCT**, non-additive aggregates — far more of SQL — but needs a **stateful runtime that keeps maintenance state the user never selects**.

The trade is explicit. The window-forward camp needs a monotonicity guarantee but no per-source change-tracking; the delta camp sidesteps monotonicity entirely but pays with a whitelist + full-refresh fallbacks (Snowflake/BigQuery) or a stateful differential runtime (Feldera). smelt’s **incremental** sits squarely in the first camp. Its **one foothold in the second camp is refresh: cumulative**: cumulative does not rebuild a window, it keeps target state and merges per-partition deltas into it — a *maintained view*, not a window rebuild. DBSP (§12.2) is the standing proof that the whitelist boundary of the first camp is a *pragmatic engineering choice*, not a fundamental limit — a fully general engine exists, at the cost of a stateful differential runtime and abandoning the “incremental $\equiv$ full over a window” simplicity smelt’s **batched** mode is built on.

The governing observation for the maintained camp, developed in Parts 13–16: **what the “smart engines” do is keep hidden maintenance state behind a clean logical relation, and smelt can do the same thing itself with a (state table + presentation view) pair**. Native IVM and smelt-emulated maintenance are then the *same logical object with two maintainers* — exactly smelt’s stated logical-spec / physical-execution separation.

1.2 The parent contract: processed-input equivalence

The two camps share one contract, worth stating once:

Processed-input equivalence. A non-full refresh produces the same result as a full refresh restricted to the inputs it has processed.

Each camp specializes it:

- **Batched** — **per-partition equivalence** (the incremental $\equiv$ full-refresh invariant, [incremental_models.md](#)): running a model as a sequence of adjacent windows must produce the same stored result, partition for partition, as recomputing the whole range in one shot. This invariant governs the entire audit of Parts 2–11.
- **Maintained** — **end-state equivalence** (§13.3, generalizing [cumulative_aggregate.md](#) §“Cross-partition equivalence”): the *user-visible* value of the model equals what a full refresh would compute over the set of inputs processed so far, independent of merge order.

Beneath the contract the camps also share machinery — `--event-time` run windows (cumulative already reuses incremental’s flags, [cumulative_aggregate.md](#) §CLI), `--auto` staleness, the derive-from-SQL posture, and `multi_backend` capability lowering. Naming the parent contract and the shared machinery once, with clearly-distinct children beneath it, is a clean documentation improvement.

It also resolves a genuine terminology collision. **The industry calls the *maintained* camp “incremental view maintenance.”** So smelt’s *incremental* (window-forward) and “incremental” in the Enzyme/Snowflake/Materialize literature name *opposite* camps. The parent term lets the docs say “both are incremental in the broad sense; here are the two shapes” — and Part 17’s `batched` rename retires the ambiguous value name entirely.

1.3 A structural umbrella would re-create the dbt footgun

The parent contract is a *conceptual* lid. Making the two camps **siblings under one selector with a strategy knob** (`refresh: incremental + strategy: window | merge`) would walk directly back into what the cumulative spec already rejected, verbatim:

“dbt conflates the two under `materialized='incremental'` and dispatches by `incremental_strategy`. This is the single most common source of confusion in dbt because the `strategy` knob silently changes the equivalence contract — same frontmatter, different invariants.” — [cumulative_aggregate.md](#) §Design

The two children differ in exactly the things that are *not* knobs:

	batched (incremental)	maintained (cumulative, ...)
output shape	partitioned, has <code>partition_column</code>	keyed lookup, no partition column
equivalence	per-partition slice	end-state
execution	window-independent (parallel, §11.2)	ordered (sequential, reads own state, §11.3)
camp (§1.1)	window-forward; needs monotone event-time	change-tracking; sidesteps monotonicity

A selector that makes those feel like variants of one strategy subordinates the single most load-bearing line in the space — stateless/window-independent vs stateful/ordered — which Part 11 is careful to keep sharp. That is a strictly worse taxonomy even though it looks tidier.

1.4 Statefulness is the spine, not the selector

The real upgrade is promoting **statefulness to the *named reason* the children differ**, rather than presenting `incremental` and `cumulative` as two arbitrary peers on a flat enum. The refresh axis reads best as:

full	– recompute everything
processed-input-equivalent	– (conceptual umbrella; §1.2)

$\vdash$ stateless / window-independent	$\rightarrow$ batched (per-partition, partitioned, parallel)
$\vdash$ stateful / maintained	$\rightarrow$ cumulative + siblings (end-state, lookup, ordered)

One caveat keeps this honest and stops it hardening into a rigid two-level selector: window-independence is a **derived** property, not a declared one (§11.4), and it *leaks* across the split — a **self-referential** batched model is stateful-ordered yet still executes as partition DELETE+INSERT, not merge_into (§11.3). So statefulness explains the *split* but must not *become* the selector; users declare a concrete mode (Part 17), and ordering stays derived.

Recommendations (ontology level). Adopt the conceptual umbrella: state processed-input equivalence as the shared parent contract, specialized to per-partition (batched) and end-state (maintained); note the shared machinery; call out the “incremental” $\leftrightarrow$ IVM terminology collision explicitly. Reject any structural/selector umbrella — no shared refresh value with a **strategy**: sub-knob. Present the refresh values in `models.md` §“Refresh axis” as processed-input-equivalent children distinguished by statefulness, not as flat peers of each other and of `full`. The value is ~90% in naming the shared contract and fixing the terminology, and actively negative in any shared selector.

Part 2 — Batched refresh: mechanism and the rejection catalogue

2.1 The mechanism, in brief

An incremental (batched) model is kept current by running one time window at a time. For each run window [run_start, run_end) the runtime:

1. **Injects a window filter on the output** — `inject_time_filter` (`crates/smelt-runtime/src/transformer.rs:272`) appends `event_time >= start AND < end` to the outermost SELECT’s WHERE (or FROM).
2. **Injects per-source scan filters** — `inject_source_filters` (`transformer.rs:65`) wraps each bounded `smelt.<path>` reference in a pushdown subquery (`SELECT * FROM smelt.<path> WHERE partition_col ...`).
3. **DELETE+INSERTs** the touched partitions of the target (or MERGEs on `unique_key`).

Correctness is the per-partition specialization of §1.2: the windowed rebuild must equal a full refresh, partition for partition. Whether a given model can be run this way is decided by a set of eligibility gates — the catalogue below — and the rest of Parts 3–11 audits each gate (and one missing gate) in turn. Part 3 works the cross-cutting question of *where* the injected filters should be written; Part 4 builds the one analysis nearly every relaxation turns out to need.

2.2 Catalogue of current rejections

There are **five enforcement pathways**, with different severity and fallback behaviour. The authoritative implementations live in `smelt-logical` (`smelt-planner` re-exports them).

Pathway A — planner safety check (`incremental::detect`)

`crates/smelt-logical/src/rules/incremental.rs`, driven from `smelt-runtime/src/execute.rs` via `check_planner_safety` (`crates/smelt-runtime/src/safety.rs`). With `enforce_safety = true` (default), a returned `Err` is a **hard error**, **build refused**. With `--allow-downgrade` (`enforce_safety = false`) it becomes a **warn!** + **silent full-table refresh**.

#	Condition	Site	Overridable via
A1	incremental: present, timeseries: missing	incremental.rs:139	(also D1)
A2	SQL not parseable by <code>analyze_select</code>	incremental.rs:146	—
A3	<code>partition_column</code> not a SELECT-list alias	incremental.rs:158	—
A4	<code>partition_column</code> not in GROUP BY (aggregate models)	incremental.rs:181	—
A5	<code>event_time_column</code> not referenced in SQL	incremental.rs:195	—
A6	<code>unique_key</code> column not a SELECT-list alias	incremental.rs:214	—
B1	window OVER whose PARTITION BY $\not\supseteq$ <code>partition_column</code>	incremental.rs:231	<code>allow_window_functions</code>
B2	HAVING	incremental.rs:248	<code>allow_having</code>
B3	LIMIT	incremental.rs:261	<code>allow_limit</code>
B4	subquery in FROM	incremental.rs:273	<code>allow_subqueries</code>
B5	non-deterministic function (RANDOM/NOW/UUID/...)	incremental.rs:288	<code>allow_nondeterministic</code>
B6	SELECT DISTINCT	incremental.rs:302	<code>allow_distinct</code>

The B-group all carry a per-model `incremental.safety_overrides.allow_*` escape hatch (`crates/smelt-core/src/config.rs:429`); the A-group do not.

Pathway C — temporal-bound derivation (`check_bound_derivation`)

#	Condition	Site	Behaviour
C1	bare LAG/LEAD (window fn without RANGE BETWEEN INTERVAL) → NotDerivable bound	incremental.rs:574 / safety.rs:100; detection analysis/source_bounds.rs:240	hard error / downgrade

Pathway D — frontmatter/metadata validation (workspace load)

#	Condition	Site	Code
D1	incremental: without timeseries:	<code>crates/smelt-core/src/metadata.rs:411</code>	<code>TimeseriesRequiredForIncremental</code>
D2	<code>partition_column</code> not projected / timeseries on ephemeral/test	<code>metadata.rs:362</code>	<code>MalformedTimeseries</code>
D3	incremental config on ephemeral / refresh: cumulative	<code>config.rs:842</code>	—

Pathway E — event-time injectability gate (`detect_built_in_rules`)

`crates/smelt-logical/src/rules/rule_diagnostics.rs`, surfaced as **Error diagnostics** through `smelt-db` → LSP and enforced at the runtime pre-execute gate. **Not** bypassed by `--allow-downgrade`.

#	Condition	Site	Code
E1	UNION/INTERSECT/EXCEPT at the outer query	<code>rule_diagnostics.rs:186</code>	<code>EventTimeColumnNotVisibleAtOuterSelect</code>
E2	bare subquery FROM not projecting <code>event_time_column</code>	<code>rule_diagnostics.rs:201</code>	<code>EventTimeColumnNotVisibleAtOuterSelect</code>

Not a rejection (recorded to avoid confusion)

- **Unbounded lookback** (`UNBOUNDED PRECEDING`, correlated subquery, default window frame) → `BatchSafety::PerPartitionOnly` (`incremental.rs:72`). The model **still builds**, just per-partition. Contrast with C1, which *does* reject.
- **IncrementalNotBatchSafe** (`rule_diagnostics.rs:154`) wraps the Pathway-A errors as a `Warning` for editor parity; it never blocks on its own.
- **Joins** — neither the inline `FROM a JOIN b` spelling nor the declared `joins:` frontmatter has **any** incremental-eligibility gate. A join model builds today with no join-specific check, and `inject_source_filters` will silently time-window *every* bounded source in the join. This is not a benign non-rejection like the two above: some join shapes are genuinely unsafe, so the missing gate is a latent soundness hole, not a conservative allowance. Worked in **Part 7**.
- **Scalar subqueries over bounded sources** (*not yet worked*) — an uncorrelated scalar subquery reading a timeseries ref (`SELECT ..., (SELECT MAX(ts) FROM smelt.silver.events) AS hwm`) has no gate (B4/E2 only look at the `FROM` clause), and because `inject_source_filters` windows every `smelt.<path>` occurrence *textually* (§6.4), the ref inside the subquery is silently time-windowed — turning a global aggregate into a per-window one, the same silent-misfilter shape as the §7.2 join hazard. (Correlated subqueries are separately caught by the unbounded-lookback → `PerPartitionOnly` path above.)
- **GROUPING SETS / ROLLUP / CUBE** (*not yet worked*) — super-aggregate rows carry `NULL` in the grouping columns, so a `GROUP BY ROLLUP(partition_col, ...)` passes a textual A4 (`partition_column` appears in the `GROUP BY`) while emitting rows whose partition column is `NULL` and whose value aggregates *all* windows — the P3 `NULL`-event-time hazard (§5.5) in aggregate form, and the classical empty-grouping-set pushdown caveat (see the PrestoDB reference).

2.3 Verdict summary

The original catalogue is fully worked; this table is the map. Each verdict is derived in the named Part via the standard frame — *why rejected* → *correctness law or mechanical limit* → *safe relaxation* → *recommendation* — and, where non-obvious, confirmed empirically (DuckDB harnesses, §5.3/§6.5/§7.5/§8.5).

Condition	Verdict	Where
E1 set operations (<code>UNION ALL</code>)	mechanical injection-point limit, not algebraic; ship the single-stream <code>UNION ALL</code> slice (every branch independently partitionable), wrap-and-filter on projected <code>event_time</code>	Part 5
B4/E2 subquery in <code>FROM</code>	injection is already correct; the real question is pushdown validity through the body — and the CTE spelling of the same query is un-gated today. Replace both gates with one parse-based body classifier applied to derived tables <i>and</i> CTEs	Part 6
Joins (un-gated)	inverse case: never gated, not universally safe. Fact ⋈ lookup is safe; a second clock (timeseries dim / second fact) or a fan-out is not. Needs driving-fact identification + fact-only source filtering	Part 7
B1 window fns + C1 LAG/LEAD + <code>UNBOUNDED</code> fallback	one cluster: frame reach. Only a bounded <code>RANGE INTERVAL</code> frame yields a derivable lookback <code>k</code> ; admit cross-partition bounded- <code>RANGE</code> windows via the primitive + derived margin; <code>ROWS/GROUPS</code> /bare <code>LAG/LEAD</code> stay rejected; <code>UNBOUNDED</code> stays per-partition (B1-regime only)	Part 8

Condition	Verdict	Where
B5 non-determinism	split the bucket: run-deterministic (<code>NOW/CURRENT_*</code>) admissible via compile-time pinning; row-nondeterministic (<code>RANDOM/UUID</code>) rejected by default, admissible via the per-column payload opt-in	§9.1, §9.2
Non-additive (holistic) aggregates	no gate warranted — the industry exclusion is an artifact of delta-style partial-aggregate merging, which smelt’s A4-aligned whole-partition rebuild never performs; the decomposability classification belongs to the maintained camp (Part 14)	§9.3
B2 <code>HAVING</code> / B6 <code>DISTINCT</code> / B3 <code>LIMIT</code>	<code>LIMIT</code> never commutes (keep gated); <code>DISTINCT</code> only when its key $\supseteq$ <code>partition_column</code> (defer to group-aligned work); <code>HAVING</code> safe-by-default when <code>GROUP BY</code> key $\supseteq$ <code>partition_column</code>	§9.4
A4 <code>partition_column</code> in <code>GROUP BY</code>	correctness law, keep; relocate its evaluation to per-branch/per-body scopes and expose its verdict as the shared partition-alignment signal	§9.5
Run-cadence $\leftrightarrow$ partition granularity	orthogonal <i>configuration</i> invariant, unchecked today: $g_run \geq g_part$ with aligned boundaries	Part 10
Window ordering / parallelism	orthogonal <i>execution</i> property, derived not declared: window-independent by default; ordered only for self-reference / cumulative shapes	Part 11

Three smaller recorded items remain unworked, each with an Open-questions entry (Part 18): scalar subqueries over bounded sources, `GROUPING SETS/ROLLUP/CUBE`, and `FOLLOWING`-frame forward reach (§8.3). Beyond those, the batched camp’s next step is not more analysis but implementation — turning the settled Parts into specs and plans; the shared first phase, the monotonicity primitive, is already built and tested (§4.8).

Part 3 — Filter placement: eligibility is pushdown depth

This part is not about one condition. It applies to every relaxation in this document — `UNION` branches (§5.4 Strategy B), subquery/CTE bodies (§6.2), and plain sources alike — because they all raise the same follow-on question: once we have *proven* a window predicate is safe, **at what depth in the query do we write it?**

3.1 Proving safety and licensing pushdown are the same fact

The eligibility proof throughout the audit is a **commutation** statement: `$\sigma_{event\_time}$` distributes over the operators between the outer select and the source (§5.2 for set ops, §6.2 for subqueries). But commutation is *exactly* the classical precondition for **predicate pushdown** in a query optimiser. So the analysis that decides “is this model incrementalisable?” is the same analysis that decides “how deep can the time filter be pushed toward the scan?” They are one computation, and it is wasteful — and, as §6.3 shows, unsound-prone — to answer the eligibility half while leaving the placement half to chance.

The practical worry: if we prove a wrapped subquery is safe and then inject the filter only on the *outer* select, we are trusting the downstream engine’s optimiser to push that predicate back down through

the derived-table (or aggregation, or CTE) boundary to the scan. If it doesn't, the model is *correct* but scans the whole source anyway — we did the proof work and handed the engine none of the benefit.

3.2 Today: two layers, two windows

The runtime already injects at two depths, for two different purposes (`execute.rs:895–913`):

1. **Outer output-clamp** — `inject_time_filter` appends `event_time >= filter_start AND < filter_end` to the **outer** select, using the **widened write window** (the run window extended backward by the derived lookback). Its job is *correctness of output*: only the current window's rows may be written.
2. **Per-source scan filter** — `inject_source_filters` wraps each `smelt.<path>` reference in `(SELECT * FROM smelt.<path> WHERE partition_col ...)` using the **un-widened run window** plus that source's derived bounds. Its job is *scan pruning*, and because it is a textual ref-replacement it already descends into subquery / CTE bodies (§6.4). Concretely it emits `partition_col >= run_start - before_secs AND < run_end + after_secs` (`transformer.rs:82–84`), and the per-source `before_secs/after_secs` come from bound derivation — so the *scan is genuinely widened*; what is “un-widened” is only the run window it starts from, before each source's own margin is added. The scan is not stuck at the bare run window.

So push-to-source is not a new idea in smelt — it already exists as the second layer. What is missing is that the two layers are derived **independently**: the outer clamp uses the widened *write* window (run window + derived lookback), while each source filter starts from the *un-widened* run window and re-derives its own `before_secs/after_secs`. Nothing guarantees the two windows agree — and code inspection shows the problem is sharper than a possible under-approximation. The two lower bounds come from **two separate analyzers over the same SQL**: `compute_effective_window` (`analysis/temporal.rs`) feeds the write window's `filter_start = run_start - k` (`windowing.rs:179–186`), while `derive_model_bounds` (`analysis/source_bounds.rs`) feeds the scan's `before_secs`; for a model with one `INTERVAL 'k'` both independently derive $\approx k$. **Equal is not enough.** The `DELETE+INSERT` covers the widened write window `[run_start - k, run_end)` — the `DELETE` is deliberately matched to the `INSERT`'s clamp for idempotency (`execute.rs:925–941`) — so every run *re-writes* the margin rows in `[run_start - k, run_start)`. Whenever the lookback reflects a genuinely cross-window reach (a cross-partition frame admitted via `allow_window_functions`, or a Form-B `WHERE/join` offset), those margin rows' own frames reach a further `k` back, to `run_start - 2k`; the scan stops at `run_start - k`, so the run recomputes them with clipped frames and **overwrites the previously-correct trailing `k` of the prior window with understated values** — the W2 under-read (§8.5), by construction, on every run. (For the B1-compliant intra-partition case the frame is truncated at the partition edge, so the rewrite is merely redundant, not wrong.) Covering the rewritten margin would need a scan margin of `2k`; the cleaner fix is the Part 8 exact-clamp design, which reads the margin but never re-writes it. Deriving both windows from one downward walk (§3.5) removes the mismatch by construction; today the *correctness* filter is the one left at the outer level, dependent on the engine to prune.

3.3 The unification: eligibility *is* maximal pushdown depth

Reframe the whole audit as a single downward walk. Starting from the outer `event_time` column, push `$\sigma_{event\_time}$` toward the sources, one operator at a time, stopping at the first operator it does **not** commute with. The point where it stops is where the filter must be written; how far it got is the eligibility verdict:

Body between outer select and source	σ pushes to...	Verdict
transparent (project / filter / rename), no lookback aggregation with GROUP BY key $\supseteq$ partition_column	the source scan — one filter both clamps output and prunes the scan just below the aggregate — the predicate is on the group key, which is a function of input columns, so it lands on the source too	safe; a single source-level filter is strictly better than the outer wrap safe; group-local
window function with a bounded RANGE lookback	genuinely two windows : a widened scan bound at the source <i>and</i> an exact output clamp above the window operator — both layers are load-bearing	safe but irreducibly two-layer
DISTINCT, LIMIT, cross-window frame, non-monotone event_time	nowhere — σ does not commute past it	reject — the pushdown wall and the eligibility wall are the same wall

Two things fall out of this table:

- For the **transparent slice** (the common subquery/CTE and single-stream UNION ALL cases), there is no lookback, so the output-clamp window and the scan window **coincide**. The two layers of §3.2 collapse into one filter, written at the source. Pushing to source is not just an optimisation here — it is the *simpler* mechanism.
- The cases that *require* the two-layer split are exactly the ones with a lookback margin (window functions), where the scan window is deliberately wider than the output window. There the outer clamp is irreducible. This tells us **when** two layers are warranted (a real lookback) versus when the second layer is just belt-and-suspenders (no lookback → the outer clamp is redundant with a source filter on the same window).

3.4 Why push at compile time rather than trust the engine

smelt’s stated identity is a **compiler and orchestrator, not a query engine** (root CLAUDE.md), and this is where that identity pays off:

- **Partition pruning needs the predicate *at the scan, on the partition column*.** On a partitioned store (Databricks/Spark Delta, Hive-partitioned Parquet), the engine prunes files only when the filter reaches the scan on the partitioning column. A predicate stranded above an aggregation or a derived table will be applied *after* a full read — correct, but unpruned.
- **Optimiser pushdown through derived tables/aggregations is not guaranteed and differs by backend.** smelt targets multiple engines; relying on each one’s optimiser to rediscover a pushdown we already proved safe makes the scan cost a function of the backend rather than of the plan. Pushing it ourselves makes the bound *guaranteed* and *portable*.
- **We already did the proof.** The commutation argument that lets us incrementalise is precisely the license to relocate the predicate. Emitting it at the source is free correctness-wise and turns the proof into an actual scan reduction.

This is the same argument as §5.4’s Strategy B (inject into each UNION branch) generalised: whenever we can prove σ commutes down to the sources, we should emit the filter *there*, and treat the outer clamp as needed only when a lookback margin makes the scan window legitimately wider than the output window.

3.5 What this implies for the work

- **Fold placement into the classifier.** The body classifier that replaces B4/E2 (§6.7) should not just return safe/unsafe — it should return the **deepest injection point** for event_time (source scan, below-aggregate, or above-window-with-lookback). Injection then writes the filter there.

- **Let the source filter subsume the outer clamp when there is no lookback.** For the transparent slice, skip the outer `inject_time_filter` wrap and rely on a source-level filter on the exact run window — fewer moving parts, guaranteed pruning. Keep both layers only when a derived lookback makes them genuinely distinct windows.
- **Unify the two bound derivations.** Today the output-clamp window and the per-source bound are computed by different code with different windows (`execute.rs:895` vs `:913`). If placement is one downward walk, the windows should be derived once, per source, from that walk. This is not just cleanup: the two windows being derived independently is the under-read of §3.2, which a single per-source derivation eliminates by construction.

3.6 Open risk

Pushing the filter to the source changes *which* rows the inner query sees, which is sound only if the pushed predicate is genuinely equivalent — the whole point of the commutation proof. The danger is a body the classifier mis-labels as transparent (e.g. a scalar subquery in the SELECT list that secretly depends on unfiltered rows, or a non-monotone `event_time` expression). The classifier must be **conservative**: when it cannot prove the outer `event_time` traces back monotonically to the source partition column, it stays at the outer clamp (today’s behaviour) or rejects — it must never push a filter it has not licensed. This is the same conservatism the empirical harness (§6.5, Q5) exists to keep honest.

3.7 Composing with a user-authored event-time filter

A model may already carry its own range predicate on the event-time — a hard floor the modeller wants on *every* run (`WHERE event_time >= DATE '2020-01-01'`), or a business rule that references it (`WHERE event_time < order_ts`). Two questions arise; both fall out of the machinery already built.

(a) **A user WHERE on the same event-time column composes by intersection.** The injected window predicate is ANDed onto whatever the outer WHERE already holds, so the effective scan is the *intersection* of the user’s range and the run window. This is correct, and — crucially — incremental $\equiv$ full is preserved *because the same user predicate constrains both the windowed rebuild and the full-refresh oracle*. A floor `event_time >= '2020-01-01'` narrows both identically; a per-window rebuild of `[ti, t{i+1})` intersected with the floor still reassembles to the full-range result intersected with the floor. A pre-existing event-time range predicate therefore never *breaks* incrementalisation. The only open choice is pushdown depth: a user predicate on the *monotone-traceable* event-time can be pushed to the source alongside the injected filter (they are conjunctive range predicates on the same column), tightening the scan further; a user predicate the classifier cannot trace (e.g. `event_time < order_ts`, a two-column comparison) stays at the outer clamp — not a hazard, merely un-pushable, exactly the §3.6 fallback.

(b) **A filter written against the *output* (post-transform) clock still resolves via the trace.** When the model projects `event_time = f(source_col)` for a monotone `f` (§4.2) and the user filters on that projected `event_time`, the Part 4 trace is what rewrites the predicate onto `source_col` for pushdown — the same `project(predicate)` move Iceberg makes (§12.3). So “the WHERE references the downstream clock, not the raw source column” is not a barrier to pushdown; it is the exact situation the monotonicity primitive exists to handle. Absent a traceable transform, the predicate is applied at the outer clamp only, and correctness still holds — it simply does not prune.

The general rule: a pre-existing event-time range predicate is always *safe* (it applies equally to the incremental sequence and the oracle); at worst it fails to push and stays above the scan.

Part 4 — The monotonicity primitive

The condition deep-dives of Parts 5–8 converge on **one shared analysis**: §5.5 (UNION branches), §3.6 (subquery/CTE pushdown), §7.4 (joins), and §8.4 (window ORDER BY keys) all block on the same predicate. This part is its full treatment. The primitive answers a single question about a model’s projected event-time:

*Does this `event_time` expression trace back, monotonically, to a real source partition column — and if so, to **which** column, on **which** source, under **what** constant offset?*

The rest of this part pins that down formally (4.1), classifies what is decidable statically (4.2), says where a static decision is impossible and a declaration is required instead (4.3), shows how the consumers call the one interface (4.4), gives its placement and shape in `smelt-logical` (4.5), enumerates the edge cases and the conservative-fallback contract (4.6), records the design decisions (4.7), and closes with what the shipped implementation established (4.8) — the primitive is **built and exhaustively tested**, awaiting its consumers. The framing is corroborated in detail by prior art — production optimisers implement exactly this analysis, and the theory names its limits — collected in **Part 12** (see especially §12.3).

4.1 Precise definition

Let a source `S` carry a partition column `p` (its `timeseries.partition_column`) and let the model project an `event_time` value through some expression `e = f(...)`. The runtime uses `event_time` in two places (§3.2): the **outer output-clamp** filters rows on the projected `e` directly (`inject_time_filter`, `transformer.rs:272`), and the **per-source scan filter** filters `S` on its partition column `p` (`inject_source_filters`, `transformer.rs:65`, using the `source_partition_col` carried by `Bound-Result::Bounded`, `source_bounds.rs:79`).

Incrementalisation is correct only when these two filters select **the same rows** — i.e. when filtering the output on `e` is *equivalent* to filtering the source on `p`. That is a statement about the function `f` relating `p` (or the source’s own event clock) to `e`.

The exact property needed is not order-preservation in the strict sense, nor value-injectivity. It is:

Interval-preimage-is-an-interval. For every window `[lo, hi)` on `e`, the set of source rows with `f(p) ∈ [lo, hi)` is exactly the set with `p ∈ [a, b)` for some thresholds `a, b` (i.e. the preimage of a half-line is a half-line). Equivalently, `f` is **monotone non-decreasing**.

Two clarifications this framing forces, both of which matter to the whitelist in 4.2:

- **Non-decreasing suffices; strict monotonicity is not required.** `DATE_TRUNC` and `CAST(ts AS DATE)` are *many-to-one* (a whole day of timestamps maps to one date) yet still push cleanly, because the model’s window boundaries are themselves granularity-aligned: `partition_column` is `DATE_TRUNC('day', e)` in the canonical model (`incremental.rs:172–176` reads the partition column’s expression `text`). A plateau of `f` never straddles a window boundary, so the half-line preimage is exact. Requiring strict monotonicity would needlessly reject the single most common shape. (This is exactly the “weakly monotone → push a **closed** source range covering the truncation bucket” rule the ClickHouse/Iceberg/Delta implementations use, §12.3.)
- **We need window-preserving, not value-preserving.** The output-clamp already filters on `e` verbatim, so it is trivially correct whenever `e` is projected (that is all `E2`’s `is_column_projected_in_sql` check, `rule_diagnostics.rs:236`, verifies today). Monotonicity is the *extra* fact required to **relocate** that filter onto `p` at the source — i.e. it licenses the Part 3 pushdown, not the bare injection. This is why the primitive is a prerequisite for the *pushdown*

half of every relaxation, and why §3.6 phrases its conservative fallback as “stay at the outer clamp” — the outer clamp needs no monotonicity, only the push does.

There is a second, weaker use where monotonicity still matters even **without** pushdown: the §5.5 *independent-partitionability* / NULL hazard. A **UNION** branch that stamps `event_time` with a constant or NULL is a *static seed*, not a monotone image of any clock — it lands in one partition forever (constant) or never passes `e >= start` at all (NULL), silently breaking incremental $\equiv$ full (property **P3**, §5.3, 1 violating row). So the predicate has to reject constant/NULL/plateau-collapsing expressions too; “monotone image of a real source clock” is precisely the condition that excludes them. The two uses share one predicate: *e is a monotone non-decreasing, total, source-traceable image of S’s clock*.

4.2 What is decidable statically from the SELECT expression

`smelt-parser` already exposes a rich typed expression tree — `Expr` offers `as_column_ref`, `as_function_call`, `as_cast`, `as_extract`, `as_case`, `as_binary` (`ast.rs:1860–1968`), `FunctionCall::name/arguments` (`ast.rs:2240,:2316`), `BinaryExpr::left/right/operator` (`ast.rs:2103–2113`), `CastExpr::expression` and its target type (`ast.rs:2725`). So a real structural classifier is feasible; it does **not** need to be a substring heuristic like the A5 test (`stripped_sql.contains(event_time_column)`, `incremental.rs:196`). One plumbing consequence, settled in §4.7: `analyze_select` retains the parsed `Expr` node on each select item so the primitive never re-parses.

Classify the event-time expression `e` by walking it from the projected column toward the leaves. The **monotone whitelist** — each form provably non-decreasing across DuckDB/Spark/Postgres, and each independently present in the ClickHouse/Iceberg/Delta whitelists (§12.3):

Form	Example	Monotone?	Traces to
transparent alias / bare column	<code>created_at AS event_time</code>	identity	the column, offset 0
qualified column	<code>f.event_ts AS event_time</code>	identity	column on the qualified input
<code>DATE_TRUNC(unit, col)</code>	<code>DATE_TRUNC('day', event_ts)</code>	non-decreasing (step)	col
<code>CAST(col AS DATE/TIMESTAMP)</code>	<code>CAST(event_ts AS DATE)</code>	non-decreasing (truncation)	col
<code>date_bin / time_bucket / FLOOR(col to grid)</code>	<code>time_bucket('1 hour', ts)</code>	non-decreasing	col
<code>col ± INTERVAL '<const>'</code>	<code>event_ts + INTERVAL '1 day'</code>	strictly increasing shift	col, offset folds into the bound
<code>col AT TIME ZONE '<fixed-offset const>'</code>	<code>ts AT TIME ZONE 'UTC', ...</code> <code>'+05:30'</code>	strictly increasing constant shift	col

This whitelist is exactly why **the output `event_time` need not equal, nor even be named the same as, any input column**. The two transformations users most often reach for are already admitted: **fixed-offset time-zone conversion** (`ts AT TIME ZONE '+10:00'`, a monotone constant shift — named DST zones are *not* monotone; see the blacklist and watch-point (c) below) and **granularity coarsening** (`DATE_TRUNC('month', ts) / CAST(ts AS DATE)`, a monotone step). Both trace back to the source clock under a folded offset, so a model that emits `business_month AS event_time` from a daily `created_at` is *Traceable*, not a special case — the primitive handles a *manipulated or renamed* output event-time uniformly. What coarsening additionally introduces — a constraint between the output partition’s *granularity* and the run cadence — is **not** a monotonicity question (the transform is monotone either way) and is worked separately in **Part 10**.

The **non-monotone / order-breaking** forms, which must yield *not-traceable*:

Form	Why it breaks	Example
arithmetic on two columns	not monotone in either alone; also multi-source (4.6)	<code>end_ts - start_ts</code>

Form	Why it breaks	Example
MOD / EXTRACT(HOUR/DOW/...)	periodic — preimage of an interval is a union of intervals	EXTRACT(HOUR FROM ts)
CASE WHEN ...	piecewise; generally neither monotone nor total	CASE WHEN ... THEN a ELSE b END
COALESCE(col, <const>)	injects a constant for NULL rows — the §5.5 seed hazard in function form	COALESCE(event_ts, '1970-01-01')
GREATEST/LEAST(col, <const>)	clamps to a plateau that <i>can</i> straddle a window boundary	GREATEST(ts, '2020-01-01')
unknown scalar UDF	monotonicity unknowable from the call site (Rice/Richardson, §12.3)	my_udf(ts)
constant / NULL literal	static seed, not a stream (§5.5 case 2)	TIMESTAMP '2020-01-01', NULL
run-nondeterministic clock	NOW()/CURRENT_DATE shift each run; not source-traceable	NOW() (also B5, incremental.rs:288)
CAST(col AS VARCHAR)	lexical order $\neq$ temporal order in general	CAST(ts AS VARCHAR)
col AT TIME ZONE '<named DST zone>'	instant $\rightarrow$ local wall clock goes backward at DST fall-back (...01:59 $\rightarrow$ 01:01...), so an interval's preimage is a union of two intervals	ts AT TIME ZONE 'America/New_York'

Where engine semantics matter. The whitelist is deliberately the intersection of what is monotone on *every* target backend, because smelt is multi-backend (§3.4) and a per-engine monotonicity table would make eligibility a function of the backend rather than of the plan. Three watch-points: (a) CAST is only whitelisted for date/timestamp target types — CAST(ts AS VARCHAR) is monotone *only* for ISO-8601 lexical form and not in general, so it is excluded; (b) month/year INTERVAL arithmetic has a non-uniform step but is still monotone non-decreasing, so it is admitted even though the offset cannot be folded to a fixed Seconds (it stays a symbolic offset — cf. source_bounds approximating MONTH $\approx$ 30 days, source_bounds.rs:506); (c) AT TIME ZONE is whitelisted **only for fixed-offset zones** ('UTC', '+05:30'). For a named zone with DST the instant $\rightarrow$ local mapping *decreases* by an hour at fall-back — confirmed empirically on DuckDB v1.4.4 (2024-11-03 America/New_York: instants one minute apart map to local 01:59 then 01:01; harness docs/research/harness/20260702-holistic_aggregate.sql, property H2) — so the preimage of a local window is a union of two disjoint intervals, not an interval. A future relaxation could admit named zones via a ± 1 h-widened scan plus an exact output clamp (the Part 8 two-layer move applied to a piecewise-monotone transform, cf. ClickHouse's factor-transformation trick, §12.3), but as a plain whitelist entry it is unsound.

Implementation note: AT TIME ZONE is not yet reachable. smelt-parser does not currently parse AT TIME ZONE syntax at all, so *neither* the fixed-offset whitelist row *nor* the named-DST blacklist row is exercised by the shipped classifier — such an expression either fails to become a SelectAnalysis item or falls through to the unrecognised-head arm, both of which yield NotTraceable. The outcome is sound (fail-closed), but the whitelist row is *aspirational*: it documents the intended verdict once the parser supports the syntax, not current behaviour. Recorded in the spec's Known Divergences.

Composition is closed under the whitelist: a composition of monotone non-decreasing functions is monotone non-decreasing, so DATE_TRUNC('day', CAST(event_ts AS TIMESTAMP)) + INTERVAL '2 hours') traces through all three layers to event_ts with a +2h offset. The classifier recurses on the single column-bearing argument at each layer and fails closed the moment a layer has two column-bearing arguments or an unrecognised head. (This is precisely the preserves_order-under-composition rule Iceberg enforces and the “a single non-monotone component poisons the chain” caveat the pushdown research draws out, §12.3.)

4.3 Where a static decision is impossible — the declared guarantee

Static classification runs out in three situations: (a) an opaque scalar UDF whose body smelt cannot see; (b) a smelt function (smelt.functions.*) whose expanded body is monotone but too large to re-derive

cheaply; (c) a genuinely data-dependent monotonicity (e.g. a column the modeller *knows* is append-only-monotone but which the SQL does not prove). For these, the safe default is *not-traceable* (4.6) — but the modeller may supply the guarantee. That the *general* case is undecidable (Rice’s theorem for arbitrary functions; Richardson’s theorem for elementary expressions, §12.3) is exactly why a declared escape hatch is unavoidable rather than a shortcut.

The natural annotation home already exists and already has this exact “trust the declaration” shape:

- **FunctionProperties** (`logical.rs:55–:64`) already carries **deterministic**, **idempotent**, **append_only** — declared, unverified booleans on a `smelt` function. A `monotone_event_time` (or per-argument `monotone`) property slots in beside them and lets a function-wrapped event-time expression be admitted by declaration when its body is not statically classifiable.
- **timeseries: frontmatter** (`config.rs:477`) is where a per-model override would live if the guarantee is about a specific model’s projection rather than a reusable function — e.g. asserting that a named event-time expression is monotone in a named source column.

The precedent — and the caution — is the declared joins: **cardinality** (`JoinSpec`, `logical.rs:103`; `Cardinality::OneToOne,OneToMany`, `logical.rs:135`). The planner already trusts that declaration for *optimisation* (join elimination, the §20E soundness caveat). A monotonicity declaration would be trusted **for correctness** — the stakes are strictly higher, exactly as §7.4 and Part 7’s open question flag. The design rule that falls out: a declaration may *widen* eligibility, but the conservative static default when no declaration is present must be *reject-the-push*, never *assume-monotone*. This matches the industry posture: every window-forward engine (Spark/Flink/dbt/ SQLMesh/cube.dev, §1.1) takes the monotone-event-time column as a *declaration* and never proves it; `smelt`’s novelty is to *prove* it where it can and fall back to declaration only where it must (§12.4).

4.4 The consumers call one interface

The condition deep-dives reduce to one call with the same signature. The input is a `SELECT` (or a `UNION` branch), the projected `event_time` expression, and the set of source refs with their declared partition columns (the `BoundContext` already built for bound derivation, `source_bounds.rs:131`; assembled from the graph in `incremental.rs:559–568`). The output is not a bare boolean — per Part 3 it must name the **deepest source column** the filter can be pushed to, so it doubles as the injection-point resolver. (The nearest production analog, ClickHouse’s `getMonotonicityForRange`, likewise returns a verdict *struct* — not a boolean — carrying direction and strictness so the caller can rewrite the range correctly, §12.3.)

- **§5.5 UNION branches** — “**independently partitionable**”. For each branch, call the primitive on that branch’s `event_time` projection against that branch’s own sources. A branch that returns *traceable* is a partitionable stream (Strategy A / B is safe on it); a branch that returns *static-seed* is the P3 NULL/constant hazard and must be named and rejected, not silently dropped.
- **§3.6 subquery/CTE conservatism**. Before pushing the proven-safe filter below a derived-table or CTE boundary, call the primitive on the outer `event_time` resolved through the body. *Traceable* → *source-column* licenses the push (Part 3 “eligibility = maximal pushdown depth”); *not-traceable* → stay at the outer clamp (today’s behaviour) — never push a filter the primitive did not license.
- **§7.4 joins** — “**exactly one input carries a monotone event_time**”. Call the primitive on the model’s `event_time` against every join input. Incrementalisable iff it returns *traceable to exactly one input* (the driving fact); that input’s scan is windowed and every other input is full-scanned. Two traceable inputs is the multi-clock hazard (J4); zero is a dim-side or ambiguous clock (reject). This replaces the A5 substring test (`incremental.rs:195`) with a resolution that names *which* input carries the clock.

- **§8.4 window ORDER BY keys.** The window-function cluster additionally needs the frame’s finite lookback margin — the primitive’s trace plus one extra returned scalar (Part 8).

The shared **output** therefore wants to be, per the Part 3 framing, a *trace* rather than a predicate — the source, the traced source column, and any constant offset — so that `inject_source_filters` can write the filter at that exact column and the offset can be merged into the derived `BoundResult` (whose `source_partition_col`, `source_bounds.rs:79`, is precisely the “deepest source column” the primitive computes). One analysis; several consumers; one injection point. This is structurally the same move as an Iceberg partition-transform `project(predicate)` — rewrite a predicate on the derived value into a predicate on the source column — recovered at compile time from the model’s SQL (§12.3).

4.5 Placement and shape in `smelt-logical`

Placement. A pure module `crates/smelt-logical/src/analysis/monotonicity.rs`, sibling to `source_bounds.rs` and `temporal.rs` under `analysis/`. This respects the **Layered single-ownership** invariant (analysis lives in `smelt-logical`, above `smelt-parser`, below `smelt-db/smelt-planner`) and the **Salsa purity** rule (a pure function over parser AST + declared context; any Salsa query in `smelt-db` is a thin wrapper that assembles the inputs and calls it). It has no new dependency — it consumes `smelt-parser`’s `Expr` tree and the existing `BoundContext`.

Shape. A trace enum plus one entry point (simplified — the shipped version additionally carries the `Monotonicity` verdict struct of §4.7):

```
/// Constant temporal shift folded out of a monotone chain (col ± INTERVAL const).
pub enum Offset { Seconds(Seconds), Symbolic(String) /* e.g. months/years */ }

pub enum EventTimeTrace {
    /// `event_time` is a monotone non-decreasing image of `source_column`
    /// on `source`, shifted by `offset`. The licence to push the filter to
    /// `source.source_column` (Part 3), and to fold `offset` into the bound.
    Traceable { source: String, source_column: String, offset: Offset },
    /// Constant or NULL-injecting – a static seed, not a partitionable stream
    /// (§5.5 case 2 / P3). Names the offending sub-expression.
    StaticSeed { reason: String },
    /// Cannot prove monotone traceability: non-monotone fn, CASE, multi-source
    /// arithmetic, unknown UDF, run-nondeterministic clock. Conservative – the
    /// consumer must not push (§3.6).
    NotTraceable { reason: String },
}

pub fn trace_event_time(
    event_time_expr: &smelt_parser::Expr,
    ctx: &crate::analysis::source_bounds::BoundContext,
) -> EventTimeTrace;
```

Why the primitive was the natural first implementation phase (and was built first, §4.8):

1. **It is the shared blocker.** §5.5, §3.6 and §7.4 cannot ship without it, and they cannot each grow a private, divergent copy without re-introducing exactly the syntax-vs-semantics inconsistency §6.3 exposed. One analysis keeps the relaxations honest with each other.
2. **It is pure and independently testable.** No injection changes, no runtime changes — a function from `(Expr, BoundContext)` to `EventTimeTrace`. It can be red-green unit-tested on

the whitelist/blacklist of 4.2 and property-tested against DuckDB (the §5.3/§6.5/§7.5 harnesses already reproduce the hazards it must catch: P3, Q5, J3–J5) *before* any consumer is wired up.

3. **Its output type is designed for the consumers, not retrofitted.** Returning a trace (source + column + offset) rather than a boolean means the same result feeds the eligibility verdict *and* the Part 3 pushdown-depth walk *and* the **BoundResult** the runtime already threads — so wiring each consumer is a small follow-on, not a re-analysis.

4.6 Edge cases and the conservative-fallback contract

- **NULL event_time (the §5.5 hazard, P3).** Any expression that can evaluate to **NULL** for some rows silently drops those rows from *every* incremental window while a full refresh keeps them. Statically this is decidable for the syntactic cases — a **NULL** literal, or **COALESCE(col, <const>)** — which the classifier routes to **StaticSeed**. It is **not** decidable at this layer for a merely *nullable column* (column nullability is inferred above **smelt-logical**, in **smelt-db**); the gate that closes this gap is a §4.7 decision.
- **Constant / static-seed event_time.** A literal timestamp → **StaticSeed** (§5.5 case 2). Distinct from a real low-volume stream (§5.5 case 1), which still traces to a genuine clock and is safe.
- **Run-nondeterministic functions.** **NOW()/CURRENT_DATE/CURRENT_TIMESTAMP** are constant-per-run but *shift between runs*, so they are not source-traceable → **NotTraceable**. This dovetails with the B5 split (§9.1): the monotonicity primitive is exactly the analysis that distinguishes a run-deterministic clock (admissible as an outer clamp, never as a pushed source filter) from a row-nondeterministic one.
- **Multi-source expression.** An **event_time** built from columns of two different sources (e.g. **f.ts** and **d.ts**) has no single source to push to → **NotTraceable**. This *is* the join multi-clock case (§7.4 / J4): the primitive returning “traceable to more than one input” is the same fact as “there is a second clock”.
- **The conservative-fallback contract (the load-bearing invariant).** The primitive must be **sound in one direction**: it may return **NotTraceable** for a form that is in fact safe (a false negative — merely a missed optimisation, the consumer stays at the outer clamp), but it must **never** return **Traceable** for a form that is not monotone-source-traceable (a false positive — an unsound pushed filter, the §3.6 danger). Every unrecognised head, every two-column argument, every unknown UDF fails **closed** to **NotTraceable**. This is the same fail-loud / fail-safe discipline the codebase already enforces elsewhere (**cardinality_from_str** maps any unknown string to the conservative **OneToMany**, **logical.rs:~146**), it is what the empirical harness (P3, Q5, J3–J5) exists to keep honest, and it is the same conservative posture every production optimiser adopts under the undecidability results of §12.3.

4.7 Design decisions

Owner decisions, implemented in [docs/plans/20260702-monotonicity-primitive-tested.md](#) (which built and exhaustively tested the primitive *before* wiring any consumer):

- **Column nullability — reject nullable leaf columns.** The pure structural trace stays in **smelt-logical** (below **smelt-db**, no type info); a thin **smelt-db** query then resolves the traced leaf column’s nullability from its own inferred schema and **downgrades Traceable → NotTraceable when the leaf is nullable-or-unknown**. The “layer on top” needed to see nullability already exists — it is **smelt-db** (it depends on **smelt-logical** and owns type inference), so no new crate is required. Syntactic **NULL** forms (**NULL** literal, **COALESCE(col, const)**) remain **StaticSeed** in the pure layer; the gate closes the *semantic* nullable-column gap.
- **Offset folding vs. symbolic offsets — carry both.** **col + INTERVAL '<const seconds/days>'**

folds into `Offset::Seconds` (merges with Form-B bound derivation); month/year intervals stay `Offset::Symbolic` for the runtime to rewrite per-engine, never silently coerced to `Seconds`.

- **Static vs. declared boundary — full static whitelist ships; declared guarantees only *widen*.** The whole 4.2 whitelist is static classification; because a declared monotonicity guarantee is trusted *for correctness* (not merely optimisation, as join cardinality is), it warrants a stricter opt-in than the existing `FunctionProperties` booleans — the exact gate (e.g. an `unstable_`-style flag) is fixed when the first declared consumer lands. **Per-backend** validation is a standing property suite (the whitelist is the intersection of what is monotone on every target backend, verified against DuckDB now and structured to add Spark/Postgres).
- **Reusing the trace as the Part 3 injection point — annotate the tree, do not track a source location.** The trace's (source, source_column, offset) is a semantic target, not a text span. The injection consumers must **annotate the logical/physical tree** with that target and let the printer emit SQL (`smelt-planner's plan_printer.rs`); they must never compute how to *edit source text*. Replacing the current textual `inject_time_filter / inject_source_filters` (`transformer.rs:65,:272`) with annotation-injection is a deferred redesign (roadmap); the primitive only guarantees its output is expressed semantically so that redesign can consume it directly.
- **analyze_select retains the Expr tree.** The primitive must not re-parse; `analyze_select` retains the parsed `Expr` on each select item (one change, many future analyses benefit). Other analyses that still re-scan raw text (clause string-scanning, `source_bounds.rs` textual `INTERVAL/RANGE` recognition, `rules/incremental.rs` `Frontmatter::strip+re-scan`, `temporal.rs` re-parse) are a roadmap cleanup sweep.
- **ClickHouse-style verdict struct — adopt the full 4-field verdict.** The traced chain carries `Monotonicity { is_monotonic, is_positive, is_always_monotonic, is_strict }` up front (alongside the three-way `Traceable/StaticSeed/NotTraceable` classification the consumers branch on). Forward-only consumers read only `is_monotonic` && `is_positive` today, but a named-DST-zone (`is_always_monotonic = false`) or descending clock (`is_positive = false`) becomes a *data* difference rather than a later type change.

4.8 What the shipped implementation established

The primitive is implemented and exhaustively tested ahead of any consumer (`crates/smelt-logical/src/analysis/monotonicity.rs`; nullability gate in `crates/smelt-db/src/queries/monotonicity.rs`; generative oracle in `crates/smelt-db/tests/monotonicity_soundness_tests.rs`; plan [20260702-monotonicity-primitive-tested.md](#)). Three facts surfaced in the build; each tightens a downstream open question rather than reopening a settled one.

- **Leaf-column resolution is name-based; no FROM/alias resolution exists at this layer.** The pure trace resolves its leaf column by matching the *name* (ignoring qualifier) against `Bound-Context.source_partition_cols`; a name that matches **zero** sources, or **more than one**, is `NotTraceable` (fail closed — `resolve_against_ctx`). There is deliberately no FROM-clause/alias machinery yet. This is sound today, but it is a concrete **prerequisite gap for the join consumer (§7.4)**: “which input carries the driving clock?” cannot be answered by name-matching when two joined sources share a partition-column name (`f.ts` vs `d.ts`) — the current primitive returns the ambiguous-match `NotTraceable`, not a driving-fact identification. Alias-scoped resolution against the model's FROM is a **new prerequisite** the join consumer must build; it is not a property the primitive already supplies. The UNION-branch and single-source subquery consumers are unaffected (each branch/body has one source scope).
- **AT TIME ZONE is currently unreachable (§4.2 implementation note).** The parser does not parse the syntax, so the whitelist's fixed-offset row is aspirational and every `AT TIME ZONE` form

fails closed to **NotTraceable**. Sound, but the eligibility surface is *narrower than the whitelist reads* until the parser is extended.

- **Per-backend validation is mechanically checked on DuckDB only.** The generative oracle compiles generated smelt models through smelt’s *own* backend codegen and searches input data for a **Traceable** verdict that breaks the output-clamp $\equiv$ source-filter commutation identity — zero counterexamples across the whitelist, and a planted-unsound arm is caught and shrunk (so the oracle provably falsifies, not passes vacuously). But **SparkOracle** today supports only type introspection, not row-level execution, so the **intersection rule** (§4.2: the whitelist is what is monotone on *every* backend) is currently *asserted by reasoning* for non-DuckDB engines and *mechanically verified* only on DuckDB. The oracle is structured for the Spark row-exec seam to drop in; until it does, a whitelist entry monotone on DuckDB but not on another backend would not be caught automatically.

A methodology note: the audit’s validation question — *property test over generated models, or a curated DuckDB fixture per deep-dive?* — is answered by this build. The generative oracle is the property-test form, it is the reusable asset the consumer plans (W2–W5 injection) build on, and the hand-written harnesses (`docs/research/harness/20260701-*.sql`) are retained as fast deterministic **seed cases** folded into its corpus.

Part 5 — Condition deep-dive: set operations at the base (E1)

5.1 Why it is rejected

Incrementalisation works by injecting `WHERE event_time >= start AND < end. inject_time_filter` (`crates/smelt-runtime/src/transformer.rs:272`) finds the **outermost** `SELECT`’s `WHERE` (or `FROM`) and appends the predicate there. On `A UNION ALL B`, a trailing `WHERE` binds to branch `B` only — branch `A` is left unfiltered, silently producing wrong data. Rather than mis-filter, `E1` (`rule_diagnostics.rs:186`, triggered by `SelectStmt::has_set_operation()`) refuses the model up front and tells the user to rewrite it by hand as a `CTE`/subquery.

So the rejection is a **mechanical limitation of the injection point**, not a statement that set operations are semantically incompatible with incremental refresh.

5.2 The correctness result

Time-filtering is a per-row predicate σ on a **projected** column (`event_time`). Selection distributes over bag-union:

$$\sigma(A \uplus B) = \sigma(A) \uplus \sigma(B)$$

and, because identical rows share every column value — including `event_time` and `partition_column`, so duplicates always fall in the same window and the same partition — the same distribution holds for **UNION** (distinct), **INTERSECT**, and **EXCEPT**:

$$\sigma(A \cup B) = \sigma(A) \cup \sigma(B) \qquad \sigma(A \cap B) = \sigma(A) \cap \sigma(B) \qquad \sigma(A \setminus B) = \sigma(A) \setminus \sigma(B)$$

The **EXCEPT** case deserves a note: a `B` row can only cancel an `A` row when the two are identical *including* `event_time` (set operations compare all projected columns, and `event_time` must be projected). So filtering `B` by the same window never removes a canceller that a full refresh would have kept — there is no cross-window cancellation hazard.

Conclusion: for all four set operations the blocker is purely how the filter is applied, not the algebra.

5.3 Empirical confirmation (DuckDB v1.4.4)

Harness: docs/research/harness/20260701-union_incremental.sql (run with duckdb -box < ...). Each property reports violating rows via $|(L \text{ EXCEPT ALL } R) \uplus (R \text{ EXCEPT ALL } L)|$; 0 = the identity holds.

Property	Violations	Meaning
P1 $\sigma(\text{big} \uplus \text{small}) = \sigma(\text{big}) \uplus \sigma(\text{small})$	0	filter distributes over UNION ALL
P2 two adjacent windows = full refresh	0	incremental $\equiv$ full for UNION ALL
P3 branch with NULL event_time	1	hazard reproduced (see §5.5)
P4a UNION (distinct) distributes	0	
P4b INTERSECT distributes	0	
P4c EXCEPT distributes	0	

5.4 Injection strategies

Strategy A — wrap and filter the projected column.

```
SELECT * FROM ( <the whole set-op body> ) AS _smelt_inc
WHERE event_time >= '<start>' AND event_time < '<end>'
```

This is literally the manual rewrite the current error suggests. It is uniform, provably correct **when event_time is projected by every branch**, and it does not care that branch 1 aliased it from created_at and branch 2 from purchased_at. It is the natural fit for the common per-row case.

- ✓ Covers merging event streams: web_events UNION ALL mobile_events.
- ✗ Fails for **aggregating** branches — SELECT date, SUM(x) ... GROUP BY date UNION ALL ... — whose output projects partition_column (date), not event_time. You cannot filter on a column that is not in the output, and filtering *after* aggregation would be wrong. These stay rejected for now.

Strategy B — inject into each branch. Walk the branch chain (SelectStmt::set_operation_select(), ast.rs:1180) and run the existing single-select injection on each branch’s WHERE, before each branch’s GROUP BY. This handles aggregating branches and per-branch differing sources, but each branch’s event_time must be independently resolvable in that branch’s FROM scope. Deferred. (Strategy B is a special case of the general “push the proven-safe filter toward the sources” argument of **Part 3**.)

5.5 Heterogeneous branches — “one side is not large”

A UNION ALL whose branches differ in source characteristics splits into three cases that look alike but are not:

1. **Small side is a genuine, low-volume timeseries source.** Correctness- identical to the symmetric case: the small branch still carries a real event_time, so Strategy A filters it exactly like the big side. “Small” only affects cost. This composes cleanly with the existing per-source pushdown, which already **skips** sources lacking timeseries: (inject_source_filters, transformer.rs:65) — so a small lookup branch is naturally full-scanned while the partition-scoped write still bounds output.
2. **Small side is a static/lookup with no real event_time (the hazard).** A UNION forces all branches to share columns, so this branch must still emit *something* in the event_time slot — a literal, or NULL.

- A **constant** timestamp lands the branch in exactly one partition, ever.
- A **NULL** makes `NULL >= start` false, so the branch **never contributes to any incremental run**, yet **does** appear in a full refresh. This silently breaks the incremental $\equiv$ full invariant. Property **P3** above reproduces exactly this: 1 violating row, the NULL-stamped branch.

3. **Small side is a dimension meant to appear in every partition.** Genuinely incompatible with partition-scoped DELETE+INSERT — a “must appear everywhere” row cannot be reconciled with a run that only rewrites the current window’s partitions. This is a **JOIN/broadcast**, **not a UNION**, and is an explicit **non-goal**.

The sharpened eligibility condition. The correct precondition for a branch is *not* “projects event_time” (the current outer-select check). It is stronger: **each branch’s event_time must be a monotone function of that branch’s own source event-time — i.e. the branch must itself be independently partitionable.** This is precisely the Part 4 primitive, called per branch (§4.4). A branch emitting a constant/NULL timestamp is a *static seed*, not a partitionable stream, and needs separate treatment (computed once into one partition), or must be rejected with a message that names the real problem.

5.6 What else must change beyond injection

Even for the narrow Strategy-A slice, deleting the E1 guard is not enough — two other gates assume a single flat SELECT:

- **incremental::detect** (`incremental.rs:132+`) runs `analyze_select` over the whole SQL; its A3–A6 checks would misfire on set-op syntax and must run **per branch**.
- **Source-bound derivation** (`analysis/source_bounds.rs`) must consider each branch’s sources when deriving pushdown bounds.

5.7 Recommendation for E1

Ship the **provably-safe, common slice** first:

- **Scope:** UNION ALL only, where **every branch is independently partitionable** (projects a real, monotone event_time).
- **Mechanism:** Strategy A (wrap-and-filter on the projected event_time), with A3–A6 lifted to run per branch.
- **Keep rejecting** (for now): aggregating-branch unions, UNION (distinct)/INTERSECT/EXCEPT, and any branch emitting a constant/NULL event-time — but replace the “rewrite this by hand” message with an honest “not yet supported” that names the specific branch/limitation.

This unlocks the pattern users actually hit — merging multiple event streams into one timeline — with a bounded, mechanical change whose correctness is both proven (§5.2) and measured (§5.3).

Part 6 — Condition deep-dive: subquery in FROM (B4 / E2)

6.1 Why it is rejected

A derived-table subquery — `SELECT ... FROM (SELECT ...) AS t` — is rejected by **two independent gates**, with different sharpness and different escape hatches:

- **B4 — planner safety, blunt, overridable** (incremental.rs:273). This is a *textual* test: if `analysis.from_text` contains a `(` and does not contain `smelt.ref( / smelt.source(`, the model is refused with “subqueries in FROM clause are not yet supported.” It carries the `allow_subqueries` override (Pathway A, B-group), so `--allow-downgrade` or a per-model `safety_overrides.allow_subqueries` turns it off.
- **E2 — event-time injectability, sharp, *not* overridable** (rule_diagnostics.rs:200). This one parses. When the outer FROM’s table expression starts with `(`, it extracts the inner SQL (`extract_balanced_parens`) and asks whether the inner SELECT projects `event_time_column` (`is_column_projected_in_sql`, honouring `*` and aliases). If not, it emits an **Error diagnostic** — surfaced in the editor via `smelt-db` and enforced at the runtime pre-execute gate. `--allow-downgrade` does **not** bypass it.

So a subquery in FROM is doubly blocked, and the two gates disagree about *what* the problem is. B4 says “subqueries, categorically, not yet.” E2 says “subqueries that hide `event_time` from the outer scope.” E2 is the real correctness statement; B4 is an older, coarser guard that predates it. The tension matters: even a subquery that *does* project `event_time` (so E2 is satisfied) is still refused by B4 unless the user reaches for the override.

Neither rejection is a statement that subqueries are semantically incompatible with incremental refresh. As with UNION (§5.1), the question is whether the time-window predicate can be applied at a point where it is (a) *visible* and (b) *equivalent to filtering the underlying source*.

6.2 Injection is already mechanically fine — the real question is pushdown validity

This is the crucial contrast with the UNION case. For UNION ALL, the blocker was the **injection point**: a trailing WHERE binds to the last branch only (§5.1). For a subquery in FROM, the injection point is **already correct**. `inject_time_filter` (transformer.rs:272) appends the predicate to the **outer** SELECT’s WHERE (or, absent one, after the outer FROM). The outer FROM is `(Q) AS t`; the injected WHERE `t.event_time >= ... AND < ...` sits *after* the closing paren and references the column the subquery projects. Mechanically, nothing is mis-scoped.

The real question is **semantic**: is filtering the *output* of the subquery Q by `event_time` equivalent to what a full refresh computes? That holds exactly when selection on `event_time` **commutes** with everything Q does:

$$\sigma_{\text{event_time}}(Q(R)) = Q(\sigma_{\text{event_time}}(R))$$

So the subquery deep-dive is not an injection-plumbing problem (as UNION was); it is a **predicate-pushdown-through-a-relational-operator** problem. The answer depends entirely on what Q contains:

Body of Q	Commutes with $\sigma_{\text{event_time}}$?	Verdict
project / rename / row filter only (transparent)	yes — selection pushes through freely	safe
JOIN where <code>event_time</code> comes from the driving (fact) side, dimensions are lookups	yes — filtering the fact side is unaffected by the lookup join	safe (same story as flat-model join pushdown)
aggregation whose GROUP BY key $\supseteq$ <code>partition_column</code> , projecting the key as <code>event_time</code>	yes — each group lives in exactly one window	safe, but <code>event_time</code> here <i>is</i> the partition key (see §6.3)
window function whose frame crosses windows, DISTINCT, LIMIT, ORDER BY+LIMIT	no — these do not commute with a row predicate	unsafe — same reasons as the flat-model B1/B6/B3 rejections, one level down

The transparent row — “I wrapped my query in a subquery for readability / to alias a computed `event_time`” — is the overwhelmingly common case, and it is provably safe. The unsafe cases are

unsafe for reasons smelt *already* names at the top level; nesting them inside a derived table does not change the algebra, only where it is written.

6.3 The CTE inconsistency (the sharpest finding)

The current E1/E2 error message tells the user to “rewrite as a CTE or subquery that projects `event_time` through all branches.” Follow that advice literally and something revealing happens.

A CTE-form query —

```
WITH t AS (SELECT ..., created_at AS event_time FROM smelt.silver.events)
SELECT * FROM t
```

— has an **outer FROM t**, which contains **no parenthesis**. Therefore:

- **B4 passes:** `from_text ("FROM t")` contains no `(`.
- **E2 passes:** the outer table expression does not start with `(`, so the subquery-projection check never runs.

The *semantically identical* derived-table form —

```
SELECT * FROM (SELECT ..., created_at AS event_time FROM smelt.silver.events) AS t
```

— is rejected by both. In SQL a CTE reference and a derived table denote the same relation; a user can convert one to the other by rote. So today’s gate keys on **syntax, not semantics**, with two consequences:

1. **The subquery rejection is largely theatre.** Any user who hits it can mechanically switch to a **WITH** and get the exact same execution — the CTE path already does what the subquery path refuses to do. We are rejecting a pattern we simultaneously allow by another spelling.
2. **If the pattern were genuinely unsafe, the CTE path would be a silent soundness hole.** The commutation question in §6.2 is identical for the CTE and the derived table. If an aggregating / windowed / **DISTINCT** body is unsafe to time-filter, the CTE form ships that unsafe query *with no gate at all* (its only backstop is E2’s set-operation check and whatever the outer SQL fails to compile). Whatever we decide for subqueries must be decided for CTEs in the same breath — they are one problem.

This is the strongest argument for replacing B4+E2 with a single **body-structure** check applied uniformly to derived tables **and** CTE bodies: classify **Q** as transparent / aggregating-aligned / order-sensitive per §6.2 and gate on *that*, not on whether a paren appears after **FROM**.

6.4 Source-bound pushdown already reaches into nested bodies

One half of incremental compilation already handles subqueries correctly today. `inject_source_filters` (transformer.rs:65) wraps each `smelt.<path>` reference in a pushdown subquery (`SELECT * FROM smelt.<path> WHERE partition_col ...`) by **textual replacement of the ref token**. Because it matches the ref wherever it appears in the SQL string, it descends into a derived table or a CTE body without caring about nesting depth — a `smelt.ref` inside `FROM (SELECT ... FROM smelt.silver.events)` gets its per-source bound just as a top-level ref would.

So the *cost-optimisation* half of incrementalisation is already nesting-agnostic. Only the *correctness window-filter* half (the outer `event_time` predicate) rejects subqueries — and, per §6.2, it rejects them for a reason that only actually applies to non-transparent bodies.

6.5 Empirical confirmation (DuckDB v1.4.4)

Harness: `docs/research/harness/20260701-subquery_incremental.sql` (run with `duckdb -box < ...`). As in §5.3, each property reports violating rows via $| (L \text{ EXCEPT ALL } R) \uplus (R \text{ EXCEPT ALL } L) |$; 0 = the identity holds. Q1 and Q4 double as the Part 3 push-to-source check — their right-hand side pushes the filter to the source (below the projection / below the aggregate) and matches the outer-clamp left-hand side exactly.

Property	Violations	Meaning
Q1 transparent body: $\sigma_e(\pi \sigma' R) = \pi \sigma'(\sigma_e(R))$	0	selection pushes through project/filter to the source (§3.3 row 1)
Q2 two adjacent windows over a transparent subquery = full refresh	0	incremental $\equiv$ full for the safe slice
Q3 derived table vs. equivalent CTE produce identical result	0	confirms §6.3 — the two spellings are one query
Q4 aggregating body, GROUP BY <code>created_at</code> $\supseteq$ <code>partition_column</code>	0	group-aligned aggregation pushes below the aggregate (§3.3 row 2)
Q5a LIMIT body — outer clamp vs. pushed	30	LIMIT does not commute with the window predicate (hazard reproduced)
Q5b cross-window frame (SUM() OVER (ORDER BY ...)) — outer clamp vs. pushed	800	an unbounded frame depends on out-of-window rows; not naively pushable (hazard reproduced)

The two hazards (Q5a/Q5b) are exactly the non-commuting bodies §6.2 flags as unsafe: their non-zero counts confirm the pushdown wall and the eligibility wall coincide (§3.3) — where the filter *cannot* be pushed is precisely where the model must *not* be silently incrementalised.

6.6 What else must change beyond the gate

As with §5.6, lifting the rejection for the safe slice touches more than the two guards:

- **B4 and E2 collapse into one body-structure classifier.** Replace the `text-contains('(')` test and the paren-prefix test with a parse-based check that (a) resolves the outer `event_time` to a subquery/CTE-projected column and (b) classifies the body as transparent / group-aligned / order-sensitive. Apply it to CTE bodies too (§6.3), closing the current CTE bypass.
- **incremental::detect's A3–A6** (`incremental.rs:132+`) run `analyze_select` over the outer query; for a derived table the `partition_column` / `unique_key` / `event_time` checks must resolve against the **subquery's** SELECT list, not the outer one that just says `SELECT *`.
- **Bound derivation** (`analysis/source_bounds.rs`) already descends textually (§6.4); confirm it still attributes bounds to the correct source when the ref is nested, and that the outer `event_time` alias is traced back to a real source partition for the window filter.

6.7 Recommendation for B4 / E2

Ship the **transparent-body slice** first, and unify the syntax split:

- **Scope:** a single derived-table subquery (or CTE body) whose body is *transparent* — projection, renaming, and row filters only, projecting a real, monotone `event_time`. This is the “wrapped for readability / to alias `event_time`” case users actually write.
- **Mechanism:** no change to `inject_time_filter` — the outer-SELECT injection is already correct (§6.2). Replace B4's text test and E2's paren-prefix test with one parse-based body classifier, applied identically to derived tables and CTE bodies (§6.3).

- **Keep rejecting** (for now): bodies containing aggregation not aligned to the partition key, cross-window window frames, `DISTINCT`, or `LIMIT` — but with an honest message that names the offending construct inside the body, not a blanket “subqueries not yet supported.” Crucially, apply the **same** rejection to the CTE spelling, so the two forms stop disagreeing.

Decision: unify on semantics, not syntax. B4 and E2 are replaced by one parse-based body classifier that resolves the outer `event_time` to a real source column and classifies the intervening operators, applied identically to derived tables and CTE bodies. The syntactic paren-test is retired. Where the proven-safe filter should then be *injected* is the Part 3 placement question.

Part 7 — Condition deep-dive: joins (the un-gated construct)

Every condition worked so far is a **rejection** the audit asks us to relax. Joins are the opposite shape: a base-relational construct that is **never rejected** for incremental eligibility, yet is not universally safe. So the deep-dive runs the same four-step frame in reverse — *why is it allowed* → *is the allowance a correctness law or an accident* → *where is it actually unsafe* → *what gate makes the safe slice safe*.

7.1 The asymmetry: audited nowhere, rejected nowhere

A join reaches smelt through two surfaces, and neither is gated:

- **Inline SQL** — `FROM fact JOIN dim ON ...` written directly in the model. `analyze_select` (`analysis/mod.rs:36`) captures the `FROM` clause as **opaque text** (`from_text`, `mod.rs:86`); it never inspects join structure. So a join model sails through every existing check: A2 (parses), A3–A6 (run on the `SELECT` list and `GROUP BY`, not the join), B4 (no `(` in `from_text` unless a subquery is also present), E1 (`has_set_operation` false), E2 (outer `FROM` does not start with `(`). Nothing looks at the join.
- **Declared joins: frontmatter** — a first-class feature (`JoinSpec`, `logical.rs:97`) describing side-joined dimensions with a **declared cardinality** (`Cardinality::{OneToOne, OneToMany}`, `logical.rs:132`) that the planner already trusts for join elimination (`EliminateUnusedLeftJoin`; the §20E declared-cardinality soundness caveat). Crucially, `joins:` is **annotation, not construction**: `planner_integration.md §7` requires each entry’s `table` to *already appear as a join alias in the body’s outermost FROM* (enforced by the `JoinsMismatch` diagnostic), its sole planner consumer is join *elimination*, and it is gated behind `unstable_schema: true`. So the declaration never injects a join into the query — it only describes, for optimisation, a join the author already wrote in SQL. Two consequences for this audit: the actual join always lives in the inline `FROM`, so the incremental gate must read the SQL and cannot rely on the frontmatter being present; and the declared cardinality is a *safety signal* the gate could reuse, but nothing wires it to *incremental* eligibility today.

This is the §6.3 pattern (a query allowed by one spelling while another is refused) taken to its limit: joins are allowed by **both** spellings, with **no** gate in either. The CTE bypass was a hole because the derived-table form was gated and the CTE form was not; the join hole is larger because *neither* form is gated at all, even though — unlike a transparent subquery — a join is not uniformly safe to time-window.

7.2 What happens today — two injections, one of them unsound

Trace an incremental join model through the two filter layers of §3.2:

1. **Outer output-clamp** (`inject_time_filter`, `transformer.rs:272`) appends `event_time >= start AND < end` to the outer `WHERE`. The column name is emitted **unqualified** (`transformer.rs` builds the predicate from the raw `event_time_column` string). On a join this has two failure modes: if the name is **ambiguous** across both sides, the engine raises a bind error — *loud*, which is tolerable; if it resolves to the **dimension** side, the model filters on a lookup’s timestamp rather than the fact stream’s clock — *silent* and wrong. The A5 guard (`incremental.rs:195`) does not catch either: it is a bare `stripped_sql.contains(event_time_column)` substring test with no scope or qualification awareness.
2. **Per-source scan filter** (`inject_source_filters`, `transformer.rs:65`) wraps **each** bounded `smelt.<path>` reference independently in `(SELECT * FROM smelt.<path> WHERE partition_col >= run_start AND < run_end)`. Sources without a derived bound (no `timeseries:`) are left untouched (`source_bounds` only emits bounds for `timeseries` refs). So:
 - a **static lookup** dimension (no `timeseries:`) is correctly full-scanned;
 - but a **timeseries dimension used as a lookup** *does* get a bound, so it is silently time-windowed on its own partition column — dropping exactly the dimension rows outside the current window that the join needs. **This is a soundness bug in an already-allowed pattern**, not a missing feature.

7.3 Correctness taxonomy of join shapes

The safety of time-windowing a join turns on **which input carries the model’s event-time clock** and whether every other input is invariant to the window:

Join shape	event_time source	Other input treated as	Window-filter safe?	Verdict
fact $\bowtie$ static dim (no <code>timeseries:</code>)	fact	full-scanned lookup (untouched)	yes — $\sigma_e(F \bowtie D) = \sigma_e(F) \bowtie D$	safe (same story as flat-model join pushdown, §6.2 row 2)
fact $\bowtie$ timeseries dim used as lookup	fact	independently windowed source (bug)	no — the dim’s own source filter drops rows the join needs	hazard — silent today
fact $\bowtie$ fact, joined on a non-partition key	one or both	independently windowed source	no in general — a fact row in window W may need a counterpart outside W	hazard — silent today
fact $\bowtie$ fact, joined on the partition key	both, aligned	co-windowed	yes — both sides share the window boundary	safe (narrow)
fact $\bowtie$ fact, equi-key plus a bounded time band (parent.ts BETWEEN child.ts — k AND child.ts)	the child (driving) fact	second fact read only within a finite band	yes — with a derived lookback k	safe (interval join, see below)
dim-side event_time	dimension		ambiguous — a lookup’s timestamp is not the stream clock	reject / at minimum loud
OneToMany fan-out	fact	row-multiplying	needs care — fan-out interacts with <code>unique_key MERGE</code>	reject unless reconciled

The declared **joins: cardinality** is precisely the discriminator between the safe rows (1:1 / lookup dimension, elidable, window-invariant) and the unsafe ones (1:N fan-out, or a second clock-bearing fact). `smelt` already trusts this declaration for join *elimination*; the same declaration should license — or refuse — time-filter pushdown to the fact side only.

The interval/temporal join is a fourth, incrementalisable shape — and the one most likely to matter in practice. Joining a child event to its *parent* on an equi-key **plus a bounded time band** — `parent_id and parent.ts BETWEEN child.ts — k AND child.ts` (equivalently `child.ts BETWEEN`

parent.ts AND parent.ts + k), the “attach the most-recent parent state to each child event” pattern — is *not* the unbounded multi-clock hazard of J4. The band gives the second fact a **finite lookback k**: the child fact is the driving clock, and the parent fact need only be scanned over [run_start – k, run_end) rather than full or (unsoundly) windowed on its own clock. This is structurally identical to the Part 8 bounded-RANGE window result — a *widened scan* on the non-driving input plus an *exact output clamp* on the driving one — and it is exactly what Flink classifies as an **append-only interval join** (§12.1, “interval/temporal = append”), as opposed to a *regular* (unbounded) equi-join which is *updating*. The safety hinge is the band: an equi-key join carrying **no** time band is the J4 multi-clock hazard (the parent counterpart can sit arbitrarily far outside the window); adding a band whose width k is a compile-time constant is precisely what makes the second clock’s reach finite and therefore derivable. So the additional join key the pattern needs — a **parent_id** equi-predicate *alongside* the time band — is allowed and expected; what the eligibility test keys on is not the number of keys but whether **exactly one** of them is a bounded temporal band against the driving clock.

7.4 The sharpened eligibility condition

A join is incrementalisable exactly when:

1. **One input is the driving fact** — it carries the model’s `event_time`, which traces monotonically back to that source’s partition column (the same “independently partitionable / monotone event-time” primitive §5.5 and §3.6 require). That primitive now exists in `smelt-logical` (§4.8), **but with a join-specific prerequisite gap**: its leaf-column resolution is name-based, so when two joined inputs share a partition-column name it returns the ambiguous-match `NotTraceable` rather than naming the driving side. The join consumer must add **alias-scoped resolution against the model’s FROM** — the “trace against every join input, exactly-one `Traceable` = driving fact” dispatch — on top of the primitive; and
2. **Every other input is a window-invariant lookup** — its contribution to any output row is independent of which window is being built. A declared 1:1 (or 1:N-lookup) dimension with no timeseries clock qualifies; a second timeseries fact joined on anything other than the shared partition key does not.

Condition 2 has a **bounded-lookback relaxation** (the interval/temporal join, §7.3): a second fact joined on an equi-key *and* a time band of compile-time width k is not window-invariant, but its read is confined to [W.lo – k, W.hi), so it is incrementalisable with a Part-8-style widened scan rather than a full one. The uniform test across all three shapes is whether the non-driving input’s contribution to window W is confined to [W.lo – k, W.hi) for a static k — invariant lookups have k = 0 (full-scanned but window-independent), band joins a finite k > 0, and an unbounded second clock (J4) no finite k at all.

Under that condition the correct injection (per Part 3) is: push the window filter to the **driving fact’s scan only**, and leave every lookup full-scanned. That is *almost* what the runtime does today for un-timeseries’d dimensions — the gap is that `inject_source_filters` wrongly also windows a *timeseries* lookup (§7.2), and that nothing identifies which single input is the driving fact.

7.5 Empirical confirmation (DuckDB v1.4.4)

Harness: `docs/research/harness/20260701-join_incremental.sql` (run with `duckdb -box < ...`). As in §5.3/§6.5, each property reports violating rows via `|(L EXCEPT ALL R) ∪ (R EXCEPT ALL L)|`; 0 = the identity holds. J1/J2 confirm the safe slice; J3–J5 reproduce the three hazards of §7.3 with a `dim_ts` whose 50 users all registered on Jan 1, well before the Jan 3–6 events that reference them.

Property	Violations	Meaning
J1 $\sigma_e(F \bowtie D_{\text{static}}) = \sigma_e(F) \bowtie D_{\text{static}}$	0	fact-side filter commutes past a static lookup (§7.3 row 1)
J2 two adjacent windows over $F \bowtie D_{\text{static}} = \text{full refresh}$	0	incremental $\equiv$ full for the safe slice
J3 $F \bowtie D_{\text{ts}}$ with the dim independently windowed	400	silent hazard of §7.2 — windowing the dim on its own clock drops every early registration row, and with it all 400 in-window event rows
J4 $F1 \bowtie F2$ on a non-partition key, both windowed	4000	multi-clock join — windowing both facts drops cross-window counterparts (the fan-out of matched pairs inflates the count)
J5 OneToMany fan-out breaks <code>unique_key</code>	400	all 400 in-window <code>event_ids</code> recur after a 1:N join, so the MERGE key is no longer unique

The three non-zero counts confirm §7.3: the safe slice (J1/J2) is a fact-side filter past a window-invariant lookup, and every shape that puts a **second clock** (J3/J4) or a **row-multiplying join** (J5) between the fact and the output breaks the incremental $\equiv$ full invariant. J3 is the one that fires on a pattern smelt **builds today** — the timeseries-dimension-as-lookup misfilter of §7.2.

7.6 What else must change beyond a gate

- **A real join detector.** `analyze_select` must stop treating `from_text` as opaque and expose join structure: the join inputs, which side each projected column (especially `event_time`) resolves to, and the join keys.
- **Qualified event-time resolution.** Replace A5’s substring test (`incremental.rs:195`) with a check that resolves `event_time_column` to a *single* input and confirms it is the driving fact — turning today’s silent dim-side misfilter into a named error.
- **Fact-only source filtering.** `inject_source_filters` (`transformer.rs:65`) must window only the driving fact, not every bounded source. A timeseries dimension used as a lookup must be recognised as a lookup and left full-scanned (or its join must be rejected). This is the concrete fix for the §7.2 bug.

7.7 Recommendation for joins

The design goal is a robust, legible eligibility model — not a patch. The three pieces below are the shape a correct implementation takes; they are independent of whether the J3 misfilter is hit by any model in flight today (smelt is early-stage — the point is that the eventual gate reasons about joins correctly, not that a live bug needs stopping).

1. **Make “which input is the driving fact” explicit in the model.** The root cause of the J3/J4 hazards is that `inject_source_filters` treats *every* bounded source as windowable, with no notion of a single clock-bearing input. The eligibility check must resolve `event_time` to exactly one input, window only that input’s scan, and full-scan every other input (§7.4). A timeseries dimension used as a lookup is then correctly full-scanned — the J3 result drops to 0 by construction.
2. **Ship the proven-safe slice (J1/J2):** `fact` $\bowtie$ (declared 1:1 / lookup) `dimension(s)`, `event_time` resolved to the fact side, per-source filter on the fact only, lookups full-scanned. This is the pattern most star-schema marts actually use, and it is exactly the safe row of the §7.5 table.
3. **Reject the unsafe shapes loudly, by name:** multi-clock `fact` $\bowtie$ `fact` joins on a non-partition key (J4), dim-side `event_time`, and OneToMany fan-out without a `unique_key` reconciliation (J5). The message must name *which input carries the clock* and *why the other cannot be windowed*, not

a blanket “joins not supported.” Legibility here is the usability win — a user who wrote a two-fact join should be told which fact smelt treats as the stream and how to express the other as a lookup.

This slots directly into Part 3: identifying the driving fact **is** the downward $\sigma_{\text{event_time}}$ push — for a join, σ commutes down to the fact scan and stops at the join for every non-fact input. The join deep-dive is therefore not a new mechanism but the join-shaped instance of the same commutation walk, blocked on the same monotonicity primitive.

Part 8 — Condition deep-dive: window functions, LAG/LEAD, and the two-layer lookback (B1 / C1)

Three catalogue entries are really **one phenomenon** seen from three angles, and §3.3 row 3 already named it: a window whose frame reaches outside the run window forces *two* load-bearing filters — a **widened scan bound** at the source and an **exact output clamp** above the window operator. B1 (incremental.rs:231, the PARTITION BY $\supseteq$ partition_column gate, overridable via allow_window_functions), C1 (incremental.rs:574 / safety.rs:100, bare LAG/LEAD $\rightarrow$ NotDerivable, detected at source_bounds.rs:240), and the UNBOUNDED PRECEDING $\rightarrow$ per-partition fallback (incremental.rs:72, a *non-rejection*) are the three faces of the same frame-reach question. This part works the cluster as a unit, in the four-step frame of Parts 5/6/7, and shows the whole cluster reduces — like §5.5, §3.6, and §7.4 — to the Part 4 monotonicity primitive plus one new quantity it must return: the **finite lookback margin** the frame reaches back.

8.1 Why it is rejected (three faces, one cause)

inject_time_filter (transformer.rs:272) writes the window predicate on the **output**; inject_source_filters (transformer.rs:65) prunes each **source scan** on the *same* run window. For every construct worked so far the two windows **coincide** (§3.3): a transparent body, a distributing UNION branch, a fact-side join filter — all need to scan exactly the rows they emit. A window function is the first construct where they legitimately **diverge**: the output row for event-time t is computed from *other* rows in its frame, so producing window [lo, hi) requires reading rows *below* lo.

The three current behaviours are three conservative responses to that divergence:

- **B1** sidesteps it by demanding PARTITION BY $\supseteq$ partition_column. When the window partitions *by* the partition column, every frame stays inside a single output partition, the two windows coincide again, and no lookback is needed — so B1’s slice is the **zero-lookback** slice and needs neither the monotonicity primitive nor an ORDER BY constraint. Anything else is gated behind allow_window_functions, i.e. refused by default.
- **C1** refuses bare LAG/LEAD because their reach is a **row** offset, and a row offset has no finite *time* bound to widen the scan by (NotDerivable, source_bounds.rs:240).
- **UNBOUNDED PRECEDING** is not rejected but degraded to BatchSafety::PerPartitionOnly (incremental.rs:72): its reach is the whole partition, so no *finite* widening recovers it — the only sound refresh is to recompute each touched partition entire.

None of the three is a statement that windowing is incompatible with incremental refresh. Each is a conservative stand-in for the missing quantity: **how far back in event-time does the frame reach, and is that distance finite and derivable?**

8.2 The correctness result: a bounded frame is a widened-scan commutation

Model the window operator as ω . Producing output window $W = [lo, hi)$ needs

$$\pi_W(\omega(R)) = \pi_W(\omega(\sigma_{[lo-k, hi)}(R)))$$

to hold, where k is the frame’s backward reach in event-time. Read literally: compute ω over a scan **widened backward by k** , then clamp the output to $[lo, hi)$. This is exactly σ/ω commutation *with a margin* — the general form of the Part 3 pushdown walk when the operator is not filter-transparent. It holds under two conditions, both supplied by the Part 4 primitive plus the frame:

1. **ORDER BY is the monotone event-time.** ω ’s frame is defined over the ORDER BY key; for a RANGE frame to correspond to an event-time interval, that key must be the model’s `event_time` (or a monotone image of it, §4.2). Then “the frame reads back k ” is a statement about event-time, and the scan bound $lo - k$ on the source **partition column** selects exactly the rows the frame reads (the same interval-preimage-is-an-interval property of §4.1).
2. **PARTITION BY $\supseteq$ partition column is *not* required** — this is the relaxation. If the window partitions by, say, `user_id` (crossing day partitions), the frame for (user u , day D) reaches into partitions $D-k \dots D$; the widened scan $[lo-k, hi)$ covers them, and the output clamp discards the widened rows so no partition outside W is rewritten. Both layers are load-bearing: drop the widening and the early-window rows are understated (proven below, **W2**); drop the clamp and rows below lo are written twice.

So the two windows differ by **exactly the lookback k** , and k is a *static* property of the frame — not a data-dependent guess. That is the whole novelty of this cluster: smelt can **derive** the margin from the SQL where streaming engines make the user declare it (§8.6).

This two-layer design is a *change* from the shipping runtime, not a description of it. Today the runtime widens **both** the outer clamp *and* the DELETE to $[run_start - k, run_end)$ (§3.2) — it *re-writes* the margin rows rather than merely reading them, and because the scan is only widened by k , the re-written margin is recomputed from clipped frames (the confirmed §3.2 under-read; a widened-write design would need a $2k$ scan). Under the exact clamp proposed here, k suffices because the margin is read, never written — and writes become strictly partition-disjoint, which is what Part 11’s parallelism claim rests on.

8.3 Frame taxonomy: only RANGE-with-INTERVAL yields a derivable time bound

The dividing line is **what the frame counts**. SQL has three frame modes, and only one measures the ORDER BY key in its own units:

Frame	Counts	Time reach k	Verdict
RANGE BETWEEN INTERVAL ' k ' PRECEDING AND CURRENT ROW, ORDER BY = event-time	value distance on the key	exactly k — the interval <i>is</i> the margin	derivable → two-layer safe
RANGE ... UNBOUNDED PRECEDING	whole partition down to $-\infty$	∞	no finite scan; per-partition recompute (BatchSafety::PerPartitionOnly)
ROWS BETWEEN n PRECEDING ...	<i>rows</i> , regardless of their event-time spacing	unbounded in time — n rows can span an arbitrary interval on a sparse stream	NotDerivable → reject (unless density declared)
GROUPS BETWEEN n PRECEDING ...	distinct peer values of the ORDER BY key	unbounded in time for the same reason	NotDerivable → reject
bare LAG(col, n) / LEAD(col, n)	one row n positions away — a degenerate ROWS n	unbounded in time	NotDerivable → reject (this <i>is</i> C1)
default frame (ORDER BY with no explicit frame)	RANGE UNBOUNDED PRECEDING AND CURRENT ROW	∞	per-partition recompute

The crucial pairing: `RANGE INTERVAL` measures the frame in the same clock the partition column is derived from, so `k` folds directly into the source bound — this is the same offset-folding `source_bounds` already does for `col + INTERVAL` (Form B, `source_bounds.rs:359`; `MONTH ≈ 30 days, :506`). `ROWS/GROUPS` count *positions*, which map to a time distance only through the data’s density — unknowable statically, hence `NotDerivable`. `LAG/LEAD` are simply the single-row case of `ROWS`, which is why C1 already lands there. And a `LAG/LEAD` carrying `no n` beyond 1 is still a *row* offset, not a time offset: the C1 rejection is correct, not merely conservative — a user with one event in the window has a predecessor arbitrarily far back (proven below, **W4**).

Forward (FOLLOWING) reach is the unworked mirror of this table. Every row above treats backward reach only. A bounded `RANGE ... INTERVAL 'a' FOLLOWING` frame (or `LEAD`, its row-offset cousin, which C1 already rejects for the same reason as `LAG`) reads rows *after* the current one, which changes the problem twice over: the scan must widen **forward** by `a` (the `after_secs` half of the source filter, `transformer.rs:83–84`, exists for exactly this — though `source_bounds` Form A currently parses `PRECEDING` frames only), and — sharper — window `W`’s output is not *settled* until the source is complete through `hi + a`: a window computed as soon as it closes will silently differ from a later full refresh once the following rows arrive. The two sound treatments are watermark-style delay (do not run `W` until `now ≥ hi + a`) or tail-rewrite (each run re-writes `[run_start – a, run_end)`, whose scan must then widen backward by `a` *plus* any preceding reach — the §3.2 composition trap again). Neither is analysed in this document yet; recorded as an open question.

`UNBOUNDED PRECEDING` deserves the sharper statement §3.3 row 3 implied: it is per-partition-recomputable **only when B1 holds** (the partition is self-contained). If the window *both* is unbounded *and* crosses the partition column, even per-partition recompute is unsound — the running total for one day depends on prior days — and the honest verdict is full recompute. The current `PerPartitionOnly` fallback is therefore correct precisely in the B1 regime and must not be extended to the cross-partition case without widening to full.

The cross-partition running total is not this cluster’s problem to solve — it is a maintained relation. An `UNBOUNDED PRECEDING` sum whose reach crosses the partition column is exactly a *cumulative aggregate*: a stored state that grows forward across every window. `smelt` already models this on a **separate refresh axis** (the D3 rejection of incremental-refresh: `cumulative` in §2.2), specified in [docs/specs/cumulative_aggregate.md](#) and executed via the `merge_into` backend primitive rather than partition-scoped `DELETE+INSERT`. So the window-cluster verdict for a genuinely cumulative reach is *not* “reject” but “**this is a maintained model, not a batched one** — use the cumulative refresh.” Where exactly that handoff line falls — and how it is surfaced to the user — is settled by the declared-mode surface (§17.6, §18.1).

8.4 The sharpened eligibility condition

A window model is incrementalisable with a **derived** lookback exactly when:

1. The **ORDER BY** key is a **monotone image of the source event-time** — the Part 4 primitive returns `Traceable{ source, source_column, offset }` for it. This subsumes B1’s implicit assumption and replaces the substring-free B1 check with the same trace the other consumers call (§4.4).
2. The frame is a bounded **RANGE** with a temporal **INTERVAL** — giving a finite `k`. `ROWS/GROUPS/LAG/LEAD/UNBOUNDED` fail this and stay on their current paths (reject or per-partition).
3. The margin composes with the source bound: `scan window = [run_start – k – offset, run_end)`; `output clamp = [run_start, run_end)`. `k` is the frame interval; `offset` is any

monotone shift the primitive folded out (§4.2). Where k is a non-uniform interval (MONTH/YEAR), it rides as the **Symbolic** offset of §4.2 / §4.7.

Under (1)–(3) the current B1 gate is **too strict** in one direction (it rejects the safe cross-partition bounded-RANGE window) and the C1/UNBOUNDED paths are **correct** (their reach is genuinely un-derivable / infinite). The relaxation is therefore narrow and precise: admit **PARTITION BY** $\not\supseteq$ `partition_column` iff the **ORDER BY** is monotone-event-time *and* the frame is bounded **RANGE**, deriving k as the second layer’s widening.

8.5 Empirical confirmation (DuckDB v1.4.4)

Harness: docs/research/harness/20260701-window_incremental.sql (run with `duckdb -box < ...`). As in §5.3/§6.5/§7.5, each property reports violating rows via `|(L EXCEPT ALL R)  $\oplus$  (R EXCEPT ALL L)|`; 0 = the identity holds. The source is a dense per-user daily grid; the model is a per-user trailing sum (**RANGE INTERVAL 2 DAY PRECEDING**, **PARTITION BY** `user_id` — deliberately *not* the partition column, the case B1 rejects today).

Property	Violations	Meaning
W1 bounded RANGE frame, scan widened by the 2-day margin	0	two-layer identity holds: widened scan + exact clamp = full refresh (§8.2)
W2 same frame, scan not widened (clamped to the output window)	80	dropping the widening understates the frame at the window’s early days (hazard reproduced)
W3 UNBOUNDED PRECEDING running total, only a finite margin applied	120	no finite widening recovers a $-\infty$ reach; must be per-partition/full recompute (§8.3)
W4 bare LAG , fixed 2-day scan margin on a sparsified stream	40	a <i>row</i> offset has no finite <i>time</i> bound; the true predecessor sits outside any fixed margin (C1 confirmed)

W1=0 is the positive result — the derived lookback makes a cross-partition window incremental $\equiv$ full. W2/W3/W4 are the three hazards the taxonomy predicts, each non-zero for exactly the reason §8.3 gives: an insufficient margin (W2), an infinite reach (W3), and a time-unbounded row offset (W4).

8.6 Prior art — everyone declares the lookback; smelt can derive it

Window-forward engines universally treat the scan margin as a **declared** number, because they cannot see the frame at plan time the way smelt can:

- **Flink** requires an **OVER** window to be **ORDER BY** the **rowtime attribute** (its monotone event-time) — precisely §8.4 condition (1) — and its **OVER** output is *append-only* once the watermark passes, versus Top-N which is *updating*. The watermark’s allowed-lateness is the declared margin; state is retained exactly that far back. Flink *validates* the **ORDER BY**-by-time requirement but never derives the reach.
- **Spark Structured Streaming** pairs `window()` with `withWatermark(col, delay)`: `delay` is the declared lateness, and windows whose end falls before the watermark are finalized and their state evicted. The margin is a user knob, not a frame derivation.
- **Databricks Enzyme** ships **WINDOW WITHOUT PARTITION BY** — analytic functions are incrementalizable **only with** a **PARTITION BY**. That is exactly smelt’s current B1 regime (zero-lookback intra-partition), externally confirming B1 as a *correct* slice while saying nothing about the cross-partition bounded-RANGE case smelt can additionally prove.

- **Snowflake Dynamic Tables** classify window functions as *blocking operators*: supported, but they push the model toward full refresh, and *non-identical PARTITION BY across window functions* blocks incremental — a coarser, cost-driven version of the same partition-alignment reasoning.
- **BigQuery MVs** exclude **all** analytic functions — the maximally conservative baseline (BigQuery does no frame analysis at all).
- **dbt microbatch** exposes `lookback = N` (reprocess the last N batches) and **SQLMesh** tracks intervals in state; both address **late-arriving source data**, a *different* axis from the frame reach (see below), and both are declared, never derived.
- **DBSP** (§12.2) makes the algebra explicit: a frame aggregation is a **non-linear** operator requiring nested integration/differentiation over the frame; a **bounded** frame is bounded state (cheaply incremental — the W1 slice), an **unbounded** frame integrates the whole partition (the W3 full-integration case). smelt’s bounded-RANGE-vs-UNBOUNDED split *is* DBSP’s bounded-vs- unbounded-state split. **Dataflow watermarks** (§12.2) are the reason a lookback exists at all: a monotone completeness bound past which late data is dropped.

Own analysis — the lookback decomposes into two independent margins, and smelt uniquely derives one of them. The scan window must be widened backward for two *orthogonal* reasons:

- **(a) computation reach** — how far the model’s own window frame reads (`RANGE INTERVAL k → k`). This is a **deterministic function of the SQL** and is exactly what smelt can derive.
- **(b) source lateness** — how late a source row may arrive relative to its event-time. This is a **data/pipeline property**, invisible in the SQL, and is what dbt `lookback` and Spark with `Watermark` declare.

Streaming engines fuse (a) and (b) into a single declared allowed-lateness because they cannot separate them. smelt can compute **(a) exactly** from the frame and needs a declaration **only for (b)** — and the two simply **add**: total scan margin = k (derived) + declared source-lateness (default 0). This is the same “prove-where-you-can, declare-where-you-must” posture as §12.4 / §4.3, now made quantitative: smelt derives the computation-reach term and leaves only the genuine data-property term to declaration. No surveyed engine makes this split; it falls straight out of smelt’s compiler-not-engine identity (§3.4).

8.7 Recommendation for the window cluster

Ship the **bounded-RANGE two-layer slice**, and fold B1/C1/the UNBOUNDED fallback into one frame-reach classifier that reuses the Part 4 primitive:

1. **Replace B1’s PARTITION BY $\supseteq$ partition_column gate with a frame-reach analysis.** Call `trace_event_time` on the window’s `ORDER BY` key; require `Traceable`. If the frame is a bounded `RANGE INTERVAL`, admit the window with a derived lookback k even when `PARTITION BY` crosses the partition column — the case B1 rejects today. The current zero-lookback intra-partition slice remains a special case ($k = 0$).
2. **Emit the two layers explicitly.** The classifier returns, alongside the Part 4 trace, the **frame margin k** ; `inject_source_filters` widens the scan to `[run_start - k - offset, run_end)` while `inject_time_filter` keeps the exact `[run_start, run_end)` output clamp. Where a declared source-lateness exists (§8.6), add it to k .
3. **Keep rejecting / degrading the un-derivable reaches, by name.** `ROWS / GROUPS` frames, bare `LAG / LEAD (C1)`, and any window whose `ORDER BY` is *not* monotone-event-time stay **NotDerivable** — but with a message that names the frame mode and says *why* a row/peer offset has no time bound, not a blanket “window functions not supported.” `UNBOUNDED PRECEDING / default` frame stays **PerPartitionOnly** when **B1 holds** and widens to full recompute when the window crosses

the partition column (§8.3).

This slots into Part 3 as the case that finally *needs* two layers: for the transparent slice the output-clamp and scan windows collapse into one filter (§3.3), but a bounded frame is the construct where they legitimately differ by exactly the derived *k* — the irreducible two-layer result §3.3 row 3 predicted, now measured (W1) and bounded (W2–W4). Like joins (§7.7), the window cluster is not a new mechanism but the frame-shaped instance of the same commutation walk, blocked on the same monotonicity primitive plus one extra returned scalar: the lookback margin.

Part 9 — Shorter conditions

Four of the catalogue’s rejections do not warrant a full deep-dive: their correctness question is settled by an argument already made elsewhere in this document, and the work is to *apply* that argument, not discover it. This part disposes of them together. Each still runs the standard frame — *why rejected* → *correctness law or mechanical limit* → *safe relaxation* → *recommendation* — but leans on Parts 3–8 rather than re-deriving.

9.1 — B5 non-determinism: split the bucket

Why it is rejected. `incremental::detect (incremental.rs:288)` rejects any model whose SQL calls a non-deterministic function — `RANDOM`, `NOW`, `UUID`, `CURRENT_DATE`, `CURRENT_TIMESTAMP`, ... — as a single class, overridable via `allow_nondeterministic`. The stated fear is the incremental $\equiv$ full invariant: a function that returns different values on re-evaluation makes a windowed rebuild disagree with a full refresh.

Correctness law or mechanical limit. The single-class treatment is a mechanical over-approximation. The functions split on *when* the value is resolved, and only one half actually breaks the invariant:

- **Row-nondeterministic** — `RANDOM()`, `UUID()`, `GEN_RANDOM_UUID()`. A fresh value *per row, per evaluation*. Re-running any window re-rolls the value, so the stored partition after an incremental run differs from a full refresh of the same range. These genuinely violate incremental $\equiv$ full **under the bit-identical contract** and stay rejected *by default* — but §9.2 lifts the rejection when the value is confined to an opted-in payload column, because a full refresh does not reproduce it either. (They are also the reason `unique_key MERGE` on a random column is meaningless — which is exactly why `unique_key` is one of the roles §9.2 keeps deterministic.)
- **Run-deterministic** — `NOW()`, `CURRENT_DATE`, `CURRENT_TIMESTAMP`, `CURRENT_USER`. Resolved *once per statement execution* and identical for every row in that run. These do not vary within a run; they vary *between* runs. Whether that breaks the invariant depends entirely on whether the incremental sequence and the reference full refresh are pinned to the *same* value.

This is the same run-vs-row distinction §4.6 already draws for the monotonicity primitive, where a run-deterministic clock is called out as “admissible as an outer clamp, never as a pushed source filter.” B5 is the projection-side twin of that rule: a run-deterministic function is safe to *emit* (it lands one constant in every row of the run), but it is never a *source-traceable* event-time and so can never license a pushed source filter — it is `NotTraceable` in the Part 4 classifier, admissible only above the scan.

Safe relaxation. Admit the run-deterministic functions by **pinning** them to a single compile-time constant, resolved once and substituted into the emitted SQL for every window of the run. The subtlety

that makes this correct — and the trap if ignored — is that the pinned value must be the value the *reference full refresh would use*, not “the wall clock at the moment each window compiles.” Concretely:

- A full refresh over $[t_0, t_N)$ evaluates `NOW()` once, to some `T_full`.
- An incremental sequence of adjacent windows $[t_0, t_1)$, $[t_1, t_2)$, ... must therefore substitute *that same* `T_full` into every window, not a per-window `now`. If each window pinned its own compile-time clock, `DATEDIFF(NOW(), event_ts)`-style expressions would drift window-to-window and diverge from the full refresh.

So the guarantee is not “`NOW()` is run-deterministic” alone — it is “`NOW()` is pinned to a single value shared across the whole incremental sequence *and* the full-refresh oracle.” Given smelt compiles the plan, it already controls this substitution point; the value belongs with the run window the runtime already threads (`execute.rs`). Row-nondeterministic functions cannot be pinned (the value is intrinsically per-row) and stay rejected.

Invocation scope — the honest limit of pinning. The pin is coherent *per invocation*. A backfill that processes many windows in one invocation shares one `T_full`, and incremental $\equiv$ full holds against a full refresh evaluated at that same instant. An *ongoing schedule* is different: each day’s run is its own invocation with its own pin, so the stored table becomes a patchwork of clocks that equals **no** single-instant full refresh — and no pinning scheme can fix that short of freezing time at the first run forever. That patchwork is usually exactly what users want from `NOW()` in a projection (`loaded_at`-style audit columns record the run that produced the row), but it must be named as a **contract change, not a preserved invariant**: pinning restores incremental $\equiv$ full *within one invocation*; across invocations, any column derived from the pinned clock is a documented divergence (or is excluded from the invariant outright).

This mirrors the industry line in §12.1: Snowflake rejects non-deterministic functions *in the SELECT projection* but permits `CURRENT_*` in a `WHERE` (where it prunes rather than materialises); Databricks Enzyme’s `EXPRESSION_NOT_DETERMINISTIC` is the same blanket B5. smelt’s refinement — pin-and-admit the run-deterministic subset — is slightly ahead of both, and cheap because smelt owns compile-time substitution.

Recommendation. Split B5 into two guards. Keep a hard rejection for row-nondeterministic functions (`RANDOM/UUID/...`). Admit run-deterministic functions (`NOW/CURRENT_DATE/CURRENT_TIMESTAMP/...`) by default, implemented as compile-time pinning to a single run-shared constant, with the invariant test being: *the same pin feeds every incremental window and the full-refresh oracle, within one invocation* (across scheduled invocations the pinned-clock columns are a documented divergence — see the invocation-scope caveat above). Retain `allow_nondeterministic` only as the escape hatch for the row-nondeterministic class. Emit an honest message naming *which* function is the problem, not “non-deterministic function” as a category.

9.2 — Opt-in non-determinism: equivalence up to full-refresh variation

§9.1 draws the row-vs-run line under the *implicit* contract that an incremental sequence must reproduce a full refresh **bit-for-bit**. But that is stricter than the invariant users actually need, and relaxing it to the real one turns the row-nondeterministic rejection from a law into a default. The motivating case: an `inserted_at = NOW()` / `loaded_at` audit stamp, or a `batch_id = UUID()` surrogate — a column the modeller is *content* to see differ, exactly as it would differ between two full refreshes.

The sharpened contract — two clauses.

1. A sequence of incremental runs must produce the same output as a full refresh.

2. Non-determinism that already differs between two different full refreshes is permitted to differ across incremental runs.

Equivalently: split the output columns into a **deterministic skeleton** and a **non-deterministic payload**. On the skeleton, incremental output is bit-identical to a full refresh; on the payload, it need only be a *plausible full-refresh draw*. A full refresh run twice already yields two different payloads, so an incremental sequence yielding a third is inside the envelope — there is nothing to preserve.

What must stay in the skeleton (never payload). A column may be payload only if its non-determinism changes *a stored value*, never *the shape* of the result. Three roles are structurally excluded, and the exclusion is a requirement of the DELETE+INSERT mechanism, not a policy choice:

- **event_time / partition_column** — decide *which window scans a row* and *which partition it is written to*. A non-deterministic clock could place a row in a different partition on rebuild; the partition-scoped DELETE+INSERT cannot reconcile that, so incremental $\neq$ full for *every* full refresh, not merely up to payload variation. This is the user’s “no non-determinism around event_time,” and the same run-vs-row exclusion §4.6 draws for the monotonicity primitive (a run-nondeterministic clock is `NotTraceable`).
- **unique_key** — decides *dedup identity*. A non-deterministic key means a window re-run cannot overwrite the rows it wrote last time (they now carry new keys), so idempotency — the property the whole DELETE+INSERT contract rests on — is lost.
- **row-set membership and grouping** — `WHERE / HAVING / JOIN ... ON / DISTINCT / GROUP BY` keys / window `PARTITION BY-ORDER BY-frame`. Non-determinism here changes *which rows exist* or *how they aggregate*, not a stored value. Two full refreshes would differ here too, but the per-window-frozen membership of an incremental build is a categorically harder object to reconcile than a payload value, so it is **out of scope** for this relaxation (Part 18).

Everything else — a projected `inserted_at = NOW()`, a `batch_id = UUID()` surrogate, a random tie-breaker stored only for audit — is payload: its value is written once per window and never consulted to place, filter, group, or dedup a row.

The relaxation. A non-deterministic function (either class, **including** the row-nondeterministic `RANDOM/UUID` §9.1 rejects outright) is admitted when its value provably flows **only** into output columns the model has *opted in* as non-deterministic, and into none of the excluded roles. The opt-in is a per-column frontmatter declaration — `incremental.nondeterministic_columns: [inserted_at, batch_id]` — and listing `event_time_column`, `partition_column`, or a `unique_key` column in it is a configuration error, since those can never be payload.

Why per-column opt-in, not a blunt flag. The existing `safety_overrides.allow_nondeterministic` disables the B5 check wholesale, dropping the skeleton guardrail with it — a random `partition_column` then sails through. The per-column opt-in *keeps* the guardrail: the modeller names exactly the payload columns they accept variation on, and the analyzer still **proves** the non-determinism did not leak into the skeleton. Non-determinism *tolerance* is inherently a declaration — only the author knows a column is audit-only, not load-bearing — the same way **deterministic/idempotent** are declared `FunctionProperties`; there is nothing in the SQL to derive it from. (This is the one place the derive-don’t-declare default correctly yields to a declaration: the fact being declared is a *value judgement about acceptable variation*, not a property of the computation.)

Enforcement is a taint check. The B5 detector (`incremental.rs:288`) already finds non-deterministic calls; the relaxation makes it *position-aware*: every such call must sit only on the RHS of a top-level projection aliased to a listed column. A call anywhere in the skeleton (or in a non-listed projection) is rejected, naming the offending position — not “non-deterministic function” as a blanket

category.

Composition with §9.1 pinning. The two mechanisms are independent axes. Pinning keeps a *run-deterministic* clock deterministic (bit-identical within an invocation) with no opt-in; the payload opt-in *lifts* the determinism requirement for a named column, covering the row-nondeterministic values pinning cannot touch. A run-deterministic clock feeding a non-listed column is still pinned; the same clock feeding a listed column may simply be left to vary. §12.1’s industry line (Snowflake/Enzyme reject non-determinism in the projection) is the blunt version; smelt’s payload opt-in is the scoped one, cheap because smelt owns the compile-time flow analysis.

Recommendation. Add `incremental.nondeterministic_columns` and gate the B5 relaxation on the taint check above; keep the blanket `allow_nondeterministic` as the discouraged escape hatch. The membership/grouping case (a non-deterministic `WHERE / GROUP BY`) goes to Part 18 — the sharpened contract would *permit* it distributionally, but reconciling frozen-per-window membership against an all-at-once full refresh needs its own argument before it is admitted.

9.3 — Non-additive (holistic) aggregates: a delta-engine rejection that does not transplant

Why it looked like a gap — and why it is not one. §12.1 obs. 2 surfaces this from the industry comparison: Snowflake Dynamic Tables and BigQuery MVs both explicitly exclude `MEDIAN`, `PERCENTILE_CONT/PERCENTILE_DISC`, and exact `COUNT(DISTINCT)` from incremental refresh, and smelt’s catalogue has no condition naming non-additive aggregates as a class. That looks like a missing **B7** gate mirroring those whitelists. **The transplant is wrong: in smelt’s refresh regime these aggregates are safe, and no gate is warranted.**

Why the industry rejection does not apply here. Snowflake, BigQuery and Enzyme are *delta* engines (§1.1): they maintain a view by **merging partial aggregates** — this refresh’s partial state combined with previously-stored state. Decomposability (a bounded partial plus an associative merge) is precisely the property *merging* needs, and holistic aggregates lack it. smelt’s window-forward `DELETE+INSERT` never merges partials: A4 (§9.5) requires the `GROUP BY` key $\supseteq$ `partition_column`, so **every group lives entirely inside one window and is recomputed from scratch, in full, by the one run that writes its partition.** There is no cross-window combination step for decomposability to matter to. A per-day `MEDIAN(latency)` with `GROUP BY day` is computed over exactly the rows a full refresh would give that group; the values are identical — including under late-data reprocessing (a re-run rewrites the whole partition from the whole group) and the Part 10 open-partition rule (the open partition is recomputed entire each run). The only way to break it is the `g_run < g_part` misconfiguration of §10.2 — which breaks `SUM` identically, so it is Part 10’s gate, not an aggregate-class gate.

Empirically (DuckDB v1.4.4, harness `docs/research/harness/20260702-holistic_aggregate.sql`): a partition-aligned `MEDIAN` over two adjacent windows vs. a full refresh — **0** violating rows, under the same $|(L \text{ EXCEPT ALL } R) \uplus (R \text{ EXCEPT ALL } L)|$ test as every other harness property.

Where decomposability *does* bite (the analysis is not wasted). The classification becomes load-bearing exactly where a **partial-merge regime** exists:

- **refresh: cumulative** (D3, [cumulative_aggregate.md](#)) maintains running state via `merge_into`. A running `SUM/COUNT/MIN/MAX` is a bounded partial; a running `MEDIAN` is not. The decomposability whitelist (`SUM/COUNT/MIN/MAX/AVG-as-SUM÷COUNT` decomposable; `MEDIAN/PERCENTILE_*/MODE/exact COUNT(DISTINCT)` holistic; unrecognised heads fail closed to holistic, §4.6) belongs to the **maintained** camp’s eligibility rules — it is exactly the algebraic ladder of Part 14 — not to the

batched catalogue.

- **Cross-window groups**, if group alignment (A4) is ever relaxed — a group spanning windows would need partial-merge to avoid re-reading other windows.
- **The MIN/MAX append-only corollary lives there too.** Within the aligned full-rewrite regime MIN/MAX need no append-only caveat (the whole group is recomputed every time). It is *merging* an extremum forward that relies on no-deletes: a delta engine (Flink, §12.1 obs. 2) must keep retraction state to recompute a MIN whose holder is deleted; a cumulative smelt model inherits the same caveat (§14.2 names why: MIN/MAX are a monoid but not a group).

Recommendation. No B7 gate for the batched regime — a partition-aligned holistic aggregate is safe and must not be rejected. Instead: (a) carry the decomposability classification into the maintained camp (Part 14 / the cumulative-aggregate spec), where merging makes it a genuine eligibility law; and (b) treat this as a methodological caution for §12.1: the industry comparison validates the catalogue only where the *refresh mechanism* behind each published rule matches smelt’s — copying a delta-engine whitelist entry into a whole-partition-rebuild regime would produce a spurious rejection.

9.4 — B2 HAVING / B6 DISTINCT / B3 LIMIT

Why they are rejected. All three are override-gated Pathway-A B-group checks (`incremental.rs:248, :302, :261`), each carrying an `allow_*` escape hatch. The question this part asks is narrower than the others: can any of the three move from *override-gated* to *safe-by-default*?

Correctness law or mechanical limit — resolved by §6.2’s commutation test. The governing fact is whether the construct commutes with the injected window predicate $\sigma_{\text{event_time}}$ (§6.2, §3.1). Run each through it:

- **LIMIT (B3) — never commutes. Keep gated.** LIMIT selects k rows from an ordered (or arbitrary) set; the k rows chosen from a single window are not the k rows chosen from the full range. $\sigma_{\text{event_time}}(\text{LIMIT}_k(R)) \neq \text{LIMIT}_k(\sigma_{\text{event_time}}(R))$. This is precisely the Q5a hazard the harness reproduces (§6.5, 30 violating rows). ORDER BY ... LIMIT (top-N) is the same wall. No safe slice; the override is the only correct path, and even then the result is not incremental $\equiv$ full — it is “the user asserts they don’t care.”
- **DISTINCT (B6) — cross-window dedup does not commute. Keep gated (with one narrow exception).** SELECT DISTINCT over columns spanning multiple windows can collapse rows that a per-window rebuild would keep separate, or vice versa. This is the non-monotone DISTINCT/GROUP BY boundary of §12.2 and the industry line of §12.1 (Snowflake/Enzyme both reject plain DISTINCT). The one case that *is* safe — DISTINCT where the dedup key $\supseteq$ `partition_column`, so duplicates can only ever fall in the same window — is the exact DISTINCT-as-degenerate-GROUP BY mirror of §6.2 row 3, and if pursued should be handled by the same group-aligned machinery as A4/§9.5, not by relaxing B6 wholesale. Absent that, keep gated.
- **HAVING (B2) — has a genuine group-aligned safe slice.** HAVING is a filter on aggregated groups. When the GROUP BY key $\supseteq$ `partition_column` (the §6.2-row-3 condition again), every group is window-local, so the HAVING predicate is evaluated over exactly the rows a full refresh would give that group — it commutes, because the filter never spans a window boundary. SELECT date, SUM(x) FROM ... GROUP BY date HAVING SUM(x) > 100 produces the same surviving groups incrementally as in full, since each date’s SUM(x) is fully materialised within its window. This is a real safe-by-default slice, gated today only because B2 is coarse.

Recommendation. Per construct:

- **B3 LIMIT** — keep as hard override-gated; there is no safe-by-default slice, and even the override is a “user accepts divergence” signal, not a correctness claim.

- **B6 DISTINCT** — keep gated by default; treat the **DISTINCT**-key $\supseteq$ `partition_column` case as part of the group-aligned aggregation work (§9.5 / A4), not a standalone relaxation.
- **B2 HAVING** — relax to safe-by-default **when** `GROUP BY` key $\supseteq$ `partition_column` (window-local groups), rejecting otherwise. This rides on the same partition-alignment check A4 already performs (§9.5) — no new analysis, just conditioning B2 on the alignment A4 establishes.

All three collapse onto one prerequisite: the partition-alignment check. B2’s safe slice and B6’s narrow exception are both “the `GROUP BY`/dedup key contains `partition_column`,” which is §9.5’s business.

9.5 — A4 `partition_column` in `GROUP BY`

Why it is rejected. A4 (`incremental.rs:181`) requires, for aggregate models, that `partition_column` appear in the `GROUP BY`. This is a genuine correctness requirement, not a conservative one: if the partition column is not a grouping key, a single output group spans multiple partitions, and a partition-scoped `DELETE+INSERT` cannot rewrite that group correctly. A4 is the check that makes “each group lives in exactly one window” (the §6.2-row-3 / §9.3 / §9.4 safe-slice precondition) *true*. It should stay.

The real work — where A4 must run. A4 today runs `analyze_select` over the **outer, flat** query (`incremental.rs:132+`). Parts 5 and 6 both flag that this location becomes wrong once aggregation can appear *inside* a construct:

- **Inside a `UNION` branch (§5.6).** §5.6 states A3–A6 “must run **per branch**.” For A4 specifically: each aggregating branch has its *own* `GROUP BY`, and each must independently include that branch’s `partition_column` projection. A branch that aggregates without partition alignment is unsafe even if its siblings are fine — A4 must be evaluated once per branch, against that branch’s `SELECT/GROUP BY`, not once over the set-op as a whole (which does not even parse as a single `GROUP BY`).
- **Inside a subquery / CTE body (§6.6).** §6.6 states A3–A6 “must resolve against the **subquery’s** `SELECT` list, not the outer one that just says `SELECT *`.” For A4: when the aggregation lives in the derived-table/CTE body (the group-aligned case of §6.2 row 3), the `GROUP BY` to check is the *body’s*, and `partition_column` must be a body grouping key traced through to the outer projection.

So A4 does not change as a *rule* — the correctness statement is unchanged — but its *evaluation site* must follow aggregation wherever the per-branch (§5.6) and per-body (§6.6) refactors relocate it. This is the same “lift the flat-model checks to the construct’s actual scope” theme both those sections raise; A4 is simply the member of A3–A6 whose relocation *also* unlocks safe slices elsewhere (B2’s **HAVING**, §9.4; **MIN/MAX** group-local aggregation, §9.3; the **DISTINCT**-key exception, §9.4). It is therefore the load-bearing one to get right.

Recommendation. Keep A4 as a correctness law. As part of the §5.6 (per-branch) and §6.6 (per-body) refactors, make A4 a **scoped** check: it takes a `SELECT` context (a branch, a subquery body, or the flat outer query) and verifies partition alignment *within that scope*. Expose its verdict (“this scope’s groups are partition-local”) as a reusable signal, since §9.4’s **HAVING** safe slice and **DISTINCT** exception condition on exactly it — and §9.3’s withdrawal of B7 rests on it too (partition-local groups are *why* holistic aggregates are safe). One partition-alignment predicate, evaluated at the right scope, several dependents — which is why this scoped check likely lands *first*, as shared infrastructure.

Part 10 — Output-time granularity and the run-window / partition alignment

Every condition worked so far asks *which column is the clock* and *whether a window filter commutes*. This part isolates a constraint that is **orthogonal to all of them** and surfaced by one concrete use case — **aggregating a daily input into a monthly output**. It is not an eligibility gate on a SQL construct; it is a relationship between three granularities that must line up for partition-scoped DELETE+INSERT to be sound, whatever the SQL looks like.

10.1 The three granularities

An incremental model has three time granularities that need not coincide:

1. **The source clock `g_src`** — how finely source rows are stamped (e.g. per- second `created_at`).
2. **The output partition `g_part`** — the granularity of `partition_column`, the unit DELETE+INSERT rewrites atomically (e.g. `DATE_TRUNC('month', ...)` → month).
3. **The run window `g_run`** — the span a single incremental run processes (the `[run_start, run_end)` the runtime threads).

Part 4’s monotonicity primitive handles the *relationship between `g_src` and the event-time transform*: `DATE_TRUNC('month', created_at)` is a monotone image of the source clock, so “emit a monthly `event_time` from a per-second source” is **Traceable** and needs no new machinery (§4.2). That is the **transform** axis, and it is fully covered. What Part 4 says nothing about is the relationship between `g_part` and `g_run` — the **cadence** axis — because it is not a property of any expression.

10.2 The constraint: a run must rewrite whole partitions

Partition-scoped DELETE+INSERT deletes an entire partition and reinserts the rows the run computed for it. That is sound **iff every partition the run touches is recomputed in full by that run** — i.e.

`g_run` must be a whole multiple of `g_part` (a run covers whole partitions), or the touched partitions must be recomputed entire (per-partition recompute).

The daily→monthly case makes the failure vivid. Partition by month, but run a *daily* window `[D, D+1)`. The run produces one day of the month’s aggregate, then DELETE+INSERT **deletes the whole month’s partition and reinserts a single day** — silently discarding the other 29 days. The invariant breaks not because `DATE_TRUNC('month', ...)` is non-monotone (it is perfectly monotone) but because the *run cadence is finer than the partition it rewrites*. The dual also fails harmlessly-but-wastefully: partition by day, run monthly — each run rewrites 30 day-partitions, which is *correct* (whole partitions) but simply a coarser cadence.

So the rule is directional — but coarseness alone is not enough: **each run’s window must cover whole partitions**, which requires *both* `g_run` a whole multiple of `g_part` *and* the run boundaries aligned to partition boundaries. A month-long run window starting mid-month is `g_run = g_part` yet spans two partial months and fails the same way as the daily run. (And when a derived lookback widens the *write* window backward, §3.2, the alignment requirement applies to the widened window’s lower edge too.) A monthly partition therefore demands month-aligned, monthly-or-coarser run windows, or the incomplete month must be handled by recompute.

10.3 The incomplete-final-partition corollary

Even with $g_run \geq g_part$, the *current* partition is a moving target: mid-month, the month’s aggregate is incomplete and will change as more days arrive. Under DELETE+INSERT this is fine **provided each run recomputes the whole month-to-date**, which $g_run \geq g_part$ already guarantees (a monthly run reprocesses the whole open month every time). The hazard is only the finer-cadence mistake of §10.2: a daily run that touches the open month writes a partial value and never revisits the earlier days. This is the same “settled vs. open window” reasoning as watermarks (§12.2) — the open partition is never settled until g_part fully elapses — applied to the *partition* granularity rather than the event-time completeness bound.

10.4 Relationship to A4 — the two alignment laws are duals

§9.5’s A4 ($partition_column \in GROUP BY$) and this constraint are **dual halves of one alignment story**:

- **A4 aligns groups $\rightarrow$ partitions.** Each output *group* must live in exactly one partition (so a partition can be rewritten without touching other groups).
- **§10.2 aligns runs $\rightarrow$ partitions.** Each *run* must rewrite whole partitions (so a partition is never left half-computed).

A4 is checked today; the run $\leftrightarrow$ partition constraint is only **half-built**. A boundary-alignment validator exists — `validate_run_window_alignment` (`smelt-runtime/src/windowing.rs:205`) checks that a run window’s boundaries land on the declared `timeseries.granularity` grid (Monday for weeks, the 1st for months, ...) — but it is not called from the live incremental path (`compute_incremental_windows`, `windowing.rs:63`, never invokes it), and `smelt` has a *single* declared granularity, so nothing cross-checks the partition-column *transform’s* unit (`DATE_TRUNC('month', ...)`) against that declaration or the run cadence. A user can declare `granularity: day`, partition by `DATE_TRUNC('month', ...)`, and run daily — the mismatch is invisible. Both alignment laws are preconditions for partition-scoped DELETE+INSERT to equal a full refresh; A4 covers the *group* side, §10.2 the *cadence* side. A complete eligibility model owes a check for the second, most naturally as a validation that the configured/derived run granularity is $\geq$ the `partition_column` granularity (both of which `smelt` can read: the partition granularity from the `DATE_TRUNC/CAST` unit the monotonicity primitive already parses, §4.2, and the run granularity from the run window the runtime threads).

10.5 Recommendation

- **Treat granularity as a validation, not an eligibility gate.** The SQL is eligible; what needs checking is a *configuration* invariant: $g_run \geq g_part$. Derive g_part from the partition-column transform unit (`DATE_TRUNC('month', ...)  $\rightarrow$  month`) via the Part 4 primitive, compare against the run cadence, and reject (or auto-coarsen the run window to g_part) when the run is finer — and wire the dormant `validate_run_window_alignment` boundary check (§10.4) into the live run path while at it.
- **Handle the open partition by recompute-of-touched-partition**, the same `PerPartitionOnly` mechanism §8.3 already uses for `UNBOUNDED` frames — the open month is recomputed entire on each run until it closes.
- **Keep this orthogonal to Part 4.** The transform (monotone image, any granularity) is Part 4’s job; the cadence relationship is a separate, cheaper check that does not touch the monotonicity classifier.

Part 11 — Window independence: run order and parallelism

Every part so far asks whether a model *can* be batched. This part asks a different, **execution-time** question the governing invariant is silent on: given that a model is incremental, **must its windows be run in order, or may they run out of order / in parallel?** The “incremental $\equiv$ full over adjacent windows” contract fixes the *result*, not the *schedule* — two orchestrators that run the same windows in different orders should both reproduce the full refresh, but only if the model permits it. The user pattern that motivates this: a model that references *previous partitions* cannot have its partitions computed concurrently or out of order.

11.1 The discriminator: does window W read the source, or its own output?

There is one clean test. Producing output window W reads some set of rows; the only question is *where those rows come from*:

- **Window-independent** — W reads only the **source** (its own window, plus a bounded lookback into *earlier source rows*, Part 8). No output window depends on the *computed result* of any other. Windows are mutually independent and may be scheduled in **any order, concurrently**.
- **Sequentially-dependent** — W reads the model’s **own prior output** or an **accumulated cross-window state**. Then W_n needs W_{n-1} to have been computed first; the windows form a chain and must run **strictly in order, no parallelism**.

Everything turns on source-vs-self, and it decides parallelism, not eligibility: a sequentially-dependent model can still be perfectly incremental — it just cannot be run out of order.

11.2 The whole safe slice of this document is window-independent

A key structural fact ties this back to Part 3 — with one load-bearing premise: **lookback must widen the source scan, never the output write**. Under the Part 8 exact-clamp design, the output clamp restricts what a run *writes* to [run_start, run_end), while any lookback margin k (Part 8 frames, the §7.3 interval-join band) only widens what it *reads* from the source. Then:

- **Writes are always partition-disjoint**. No run ever writes outside its own window, so two concurrent runs touch disjoint partitions — the DELETE+INSERT / unique_key MERGE of different windows never collide.
- **Overlapping reads are harmless**. A lookback makes adjacent windows’ *source* scans overlap, but a read-read overlap on the immutable source imposes no ordering.

Today’s runtime does not yet satisfy the premise. Both the outer clamp and the DELETE currently use the *widened* write window [run_start - k, run_end) (§3.2), so whenever a lookback is derived, adjacent windows’ write ranges overlap by k — two concurrent adjacent runs would DELETE+INSERT overlapping partitions. Window-independence therefore holds unconditionally only for zero-lookback models today, and extends to lookback models once the exact-clamp design lands (or with k-separated scheduling in the interim). The same caveat attaches permanently to any **declared source-lateness margin** (§8.6 axis (b)): its whole purpose is to *re-write* earlier partitions on later runs, so late-data reprocessing makes adjacent runs’ writes overlap *by design* — a model using it trades out-of-order freedom for lateness tolerance exactly where the margin applies.

Consequently **every relaxation worked in this document — transparent subquery/CTE (Part 6), UNION ALL streams (Part 5), fact $\bowtie$ lookup and the bounded interval join (Part 7), and the bounded-RANGE window (Part 8) — is window-independent, given exact output clamps**. Each reads only source rows (its window $\pm$ a source-side margin) and writes only its own

partitions. All of them may be run out of order and in parallel. This is not a coincidence: it is the same monotone/linear frontier (§12.2) — the operators that commute with the delta are exactly the ones whose per-window output does not depend on other windows.

11.3 What forces sequential execution

Only shapes that read *computed* cross-window state are sequential, and they are already named elsewhere in this document:

- **Cumulative aggregates** (refresh: `cumulative`, D3, `cumulative_aggregate.md`) — the running total for window *W* is the accumulated state through *W*−1. Inherently ordered; this is the defining reason cumulative sits on its own refresh axis and uses `merge_into` rather than partition-scoped DELETE+INSERT. In the taxonomy of this document, it is the first member of the **maintained** family (Parts 13–16).
- **Self-referential incremental models** — a model whose SQL reads its *own* prior partitions (`smelt.ref` to itself, or an engine-level read of the target table for `partition < current`). Each window consumes the last, so the chain is strict. This is the pattern the motivating user names directly, and it is the incremental cousin of the cumulative case — the dependency is on *own output* rather than a maintained aggregate, but the ordering consequence is identical.
- **Cross-partition UNBOUNDED PRECEDING** windows (§8.3) — the reach that §8.3 already routes to per-partition/full recompute; when it genuinely accumulates across partitions it is the cumulative case above.

11.4 Derived, not declared

The “must run in order” property should **fall out of analysis, not a frontmatter knob** (the derive-don’t-declare posture the rest of this audit takes). Both sequential triggers are statically visible: a **self-reference** is a property of the model’s ref graph (does the model’s dependency set include itself?), and a **cumulative/cross-partition-unbounded** shape is exactly what the refresh axis (D3) and the Part 8 frame classifier already detect. So a model is window-independent *by default*, and becomes sequential only when the graph shows a self-edge or the refresh axis is **cumulative**. No new declared property is needed; window-independence is the derived complement of “reads its own output.”

11.5 Prior art — independence is *why* engines parallelise batches

The split is externally load-bearing, not merely theoretical:

- **dbt microbatch runs batches concurrently** precisely because it treats each batch as independent (each reads its own `event_time` slice of the source); it exposes batch parallelism as a first-class knob. That is the window-independent case made operational.
- **SQLMesh** tracks intervals and can backfill independent intervals in parallel for the same reason.
- **Streaming engines** draw the opposite line for stateful operators: a running aggregate keeps ordered state (§12.2 watermarks) — the sequential case — while stateless map/filter/append stages are embarrassingly parallel (DBSP’s *linear* operators, §12.2). `smelt`’s window-independent slice is the batch analog of a stateless streaming stage; its cumulative/self-referential slice is the stateful one.

11.6 Recommendation

- **Name the property and derive it.** Add a derived model property — *window-independent* vs *ordered* — computed from (a) the ref graph (self-edge → ordered) and (b) the refresh axis

/ Part 8 frame reach (cumulative or cross-partition-unbounded $\rightarrow$ ordered). Default is *window-independent*.

- **Let the orchestrator use it.** A window-independent model may have its runs scheduled out of order and in parallel (disjoint-partition writes make this safe, §11.2); an ordered model must be run as a strict forward sequence. This is an execution-planning signal, adjacent to but distinct from the eligibility gates — a model can be eligible *and* ordered.
- **Surface ordering in diagnostics.** When a model is *ordered*, say *why* (self reference / cumulative), so a user who expected parallel backfill understands the constraint — the same legibility posture as the join and window rejections.

Part 12 — Prior art and external validation

The batched-camp audit reasons from first principles plus an empirical DuckDB harness (§5.3/§6.5/§7.5/§8.5). This part checks that reasoning against three external bodies of work: the **academic theory** of incremental computation, the **published eligibility rules** of production incremental-view/materialized-view engines, and the handful of systems that already implement something like the **monotonicity primitive** (Part 4). The headline: every load-bearing claim has independent support, smelt’s rejection catalogue (Part 2) is reproduced near-item-for-item by the systems that publish theirs, and smelt’s one genuinely novel ambition is to *infer and prove* the monotone-event-time property that every comparable window-forward system instead asks the user to *declare*. Full citations are in **References**. (The two-camps split this validation rests on is §1.1.)

12.1 The catalogue is externally validated

Every *whitelist* engine that publishes its rules — Snowflake Dynamic Tables, BigQuery MVs, Databricks Enzyme, and (as a changelog “append vs updating” type) Flink — independently reproduces smelt’s rejection catalogue (Part 2) almost item-for-item. This is strong evidence the catalogue is *correct*, not merely conservative. Databricks Enzyme even ships a named error class, `MATERIALIZED_VIEW_NOT_INCREMENTALIZABLE`, whose sub-conditions map onto smelt’s directly: `EXPRESSION_NOT_DETERMINISTIC` (B5), `WINDOW_WITHOUT_PARTITION_BY` (exactly smelt’s B1 `PARTITION BY  $\supseteq$  partition_column` rule), `AGGREGATE_NOT_TOP_NODE` (the “aggregate must be the outer node” constraint behind §6.2 row 3), and `SUBQUERY_EXPRESSION_NOT_INCREMENTALIZABLE` (B4/E2).

Legend: ✓ incremental-safe · ✗ unsupported $\rightarrow$ full recompute/reject · ~ conditional · — not separately documented.

smelt rejection	Snowflake Dynamic Tables	BigQuery MV	Databricks Enzyme	Spark Structured Streaming	Flink (changelog)	Materialize	dbt / SQLMesh
UNION ALL (E1)	✓	~ preview	✓ (row tracking)	✓ stateless	✓ (least-monotone branch) updating	✓	trusted
UNION/INTERSECT/EXCEPT (distinct)	✗ INTER- SECT/EXCEPT	✗	✗ plain UNION	✗ (needs dedup)		✓	trusted
subquery-in-FROM (B4/E2)	✓ in FROM; ✗ outside	✗ ARRAY subq	✗ scalar/expr; FROM/CTE ✓	—	✓	✓	trusted
joins (Part 7)	~ OUTER + equality only	~ INNER✓, non-leftmost change $\rightarrow$ full	~ in- ner/L/R/full ✓; ✗ cross/semi/anti	~ stream=fact; outer needs water-mark+range	regular = updating; interval/temporal = append	✓	trusted

smelt rejection	Snowflake Dynamic Tables	BigQuery MV	Databricks Enzyme	Spark Structured Streaming	Flink (changelog)	Materialize	dbt / SQLMesh
DISTINCT (B6)	✓	✓ (no exact COUNT(DISTINCT))	✗ plain DISTINCT	✗ (dropDuplicatesWithWatermark)	dedup = ✓	✓	trusted
window fns (B1/C1)	✓ mostly	✗ all analytic	~ ✓ only w/ PARTITION BY	— (event-time window())	OVER = append; Top-N = updating	✓	trusted
non-deterministic (B5)	✗ in SELECT (✓ in WHERE)	✗ RAND/CURRENT_*	✗ EXPRES-SION_NOT_DETERMINISTIC	✗	✗	✗	trusted
HAVING (B2)	✓	✗ (non-incr only)	✓	—	via updating agg	✓	trusted
LIMIT (B3)	✗ LIMIT/TOP	—	✗	✗	Top-N = updating retraction state	✓	trusted
non-additive agg (no smelt gate — see §9.3)	✗ MEDIAN/PERCENTILE_COUNT(DISTINCT)	✗ exact	—	group-by = updating	✓	✓	trusted
verifies ≡ full?	✓ fails CREATE	✓ whitelist	✓ algebraic delta or full	✓ engine	✓ engine	✓ engine	✗ trusts user

Three observations fall out of the table:

1. **The UNION-vs-UNION ALL split (§5.2) is industry-standard.** Snowflake (“UNION = UNION ALL + SELECT DISTINCT”), BigQuery, and Enzyme all draw exactly smelt’s line — bag-union distributes, distinct-union drags in a DISTINCT that does not. smelt’s algebraic argument (§5.2) is the same fact these engines encode as a whitelist entry.
2. **An apparent gap that turned out not to transplant: non-additive aggregates.** Snowflake and BigQuery both explicitly exclude MEDIAN, PERCENTILE_CONT/DISC, and exact COUNT(DISTINCT) — they depend on *all* rows, not just the window’s. smelt covers DISTINCT (B6) but names no such class, which looks like a candidate new condition. §9.3 works it and finds the exclusions are artifacts of **delta-style partial-aggregate merging**, which smelt’s A4-aligned whole-partition rebuild never performs — in smelt’s regime these aggregates are safe, and the classification matters only for the maintained camp (Part 14). The general caution this yields: the table validates the catalogue only where the refresh *mechanism* behind a published rule matches smelt’s. Corollary: MIN/MAX are additive-enough that Snowflake *supports* them, but they are non-monotone under *deletes* — merging extrema forward relies on append-only, where a delta engine (Flink) must keep retraction state.
3. **Eligibility vs. cost (from Enzyme).** Databricks decouples “is this incrementalizable?” from “*should* we” — even when incrementalizable, a cost model may still pick full recompute (e.g. large source deletes). This is the reference design if smelt ever wants a “fall back to full-window recompute rather than hard-reject” mode, mapping onto smelt’s existing --allow-downgrade posture.

12.2 The theory names smelt’s safe slice exactly

The empirical safe-slice/hazard split of Parts 5–8 is not a coincidence — it is the classical **monotone** / **non-monotone frontier** of database theory:

- **Monotone = incrementally-maintainable-without-recompute.** A query is monotone iff $I \subseteq I' \implies Q(I) \subseteq Q(I')$ (input growth only adds output, never retracts). The **CALM theorem** (Hellerstein’s 2010 conjecture; Ameloot–Neven–Van den Bussche’s PODS 2011 / JACM 2013 proof) sharpens this to *iff*: monotone queries are **exactly** the add-only, coordination-free class. smelt’s window-by-window refresh *is* an add-only computation over a growing event stream, so

“incremental $\equiv$ full-refresh” holds by construction precisely for the monotone slice — and *only* there.

- **The operator boundary is the doc’s boundary.** Positive relational algebra (σ , π , $\bowtie$, $\cup$) is monotone; aggregation, EXCEPT/negation, and DISTINCT/ GROUP BY are non-monotone (a new or late row can retract prior output). This is exactly smelt’s safe-slice-vs-hazard split, now grounded in theory rather than the harness.
- **Eligibility proof = pushdown license (one fact).** smelt’s commutation statement $\sigma \circ Q = Q \circ \sigma$ (§3.1) is the classical precondition for predicate pushdown (System R 1979; Garcia-Molina–Ullman–Widom §16.2). So “is it incrementalisable?” and “how deep can the filter push?” are one computation — the Part 3 unification is textbook-sound, not a smelt invention.
- **The modern constructive statement: DBSP** (Budiu et al., VLDB 2023 Best Paper). The incremental form of any query is $Q^{\Delta} = D \circ Q \circ I$, provably equal to recompute. Its cost taxonomy is smelt’s taxonomy: **linear** operators (select/project/map/union) satisfy $Q^{\Delta} = Q$ — the cheap safe slice that commutes with the delta; **bilinear** = joins (product rule); aggregation/negation need nested integ/differentiation. DBSP’s linear-vs-bilinear split is the precise algebraic statement of the doc’s “safe slice.”
- **The streaming analog: watermarks** (Akidau et al., Dataflow, VLDB 2015): a monotone lower bound on event-time completeness that “only moves forward” — the mirror of smelt’s bounded-lookback reasoning and the reason late/out-of-order data forces lookback.

The provable limits the design must respect. The invariant cannot be decided in general:

- **Query equivalence for full SQL is undecidable** (Trakhtenbrot, via standard reduction) — so “incremental plan $\equiv$ full-refresh plan” cannot be decided by comparing arbitrary models. Even monotone Datalog has undecidable equivalence: monotonicity buys *maintainability*, not *decidable equivalence*.
- **Monotonicity of an arbitrary function/UDF is undecidable** — Rice’s theorem in general; Richardson’s theorem (1968) already for elementary real expressions built from $+$ $-$ $\times$ $\circ$, $\sin$, $\exp$, abs . So the primitive *cannot* auto-prove monotonicity of an opaque `event_time` expression.
- **Traction is regained only on restricted fragments:** conjunctive-query equivalence is decidable (NP-complete, Chandra–Merlin 1977), and monotonicity is soundly-but-incompletely decidable over a whitelist (positive relational algebra — order-preserving scalar constructs).

This answers the recurring question — *how much can be decided statically vs. needs a declared guarantee?* — crisply: **the primitive must be a sufficient-condition analysis** (decide monotonicity soundly over a whitelist, require a declaration everywhere else, never push an unlicensed filter), exactly the §3.6 / §4.6 conservative contract.

12.3 The monotonicity primitive already exists — in three other shapes

smelt’s Part 4 primitive is not speculative: three production systems implement a close analog, and their designs directly inform §4.2–4.5.

1. **ClickHouse IFunctionBase::getMonotonicityForRange** — the one production engine that reasons about function monotonicity *at plan time* to push a predicate on a derived expression onto a sorted source key. It returns a four-boolean verdict `Monotonicity { is_monotonic, is_positive (direction), is_always_monotonic, is_strict }` per function per range, consumed by `KeyCondition` to rewrite a predicate on `toStartOfDay(ts)/toDate/CAST` into a predicate on the primary key. **This is the closest structural analog to the verdict Part 4’s classifier returns** — and the argument (§4.4, §4.7) for returning a verdict rather than a boolean. Altin-

ity’s write-up documents the *factor-transformation* trick for piecewise-monotone date functions (`toMonth`, `toDayOfWeek`) — monotone only within a range over which a coarser factor is constant — a technique smelt could reuse for EXTRACT-style expressions.

2. **Apache Iceberg partition transforms** — the cleanest *formalization*. Each transform carries a `preserves_order` boolean (`year/month/day/hour`, `truncate`, `identity` = true; `bucket`, `void` = false) plus a `project(predicate)` method: order-preserving transforms project a *range* predicate onto the partition field; `bucket` (a hash) can project *equality only*. **This is precisely smelt’s licensing rule** — a range-predicate pushdown is sound iff the derivation is order-preserving — and smelt’s static primitive is essentially a compile-time `preserves_order` classifier + `project`-style rewriter over `event_time = f(source_col)`.
3. **Delta Lake generated columns** — the inverse layout (partition column is the derived one), identical condition: a fixed whitelist of order-preserving generation expressions (`CAST(ts AS DATE)`, `YEAR/MONTH/DAY/HOUR`, prefix `DATE_FORMAT`, `SUBSTRING`) whose inverse image of a range is a range, letting a query on the source column produce a partition filter. A battle-tested enumeration smelt’s whitelist (§4.2) can mirror.

The negative baseline sharpens Part 3’s “push at compile time, don’t trust the engine” thesis (§3.4): Oracle, PostgreSQL, and SQL Server deliberately do *not* reason about monotonicity — *any* function wrapping the partition key defeats pruning, and the sanctioned workaround is to materialize the transform as a virtual/generated column. DuckDB (a smelt target) exploits a source-column range via zonemaps *once smelt has done the monotone rewrite*, but will not derive that rewrite from a derived-column predicate itself. So among common backends only ClickHouse would do this for you — and smelt is multi-backend, which is exactly why the rewrite must be smelt’s job. All of them bottom out on a **hard-coded whitelist** because the general problem is undecidable (§12.2) — the same conservative posture §4.6 adopts.

12.4 Where smelt is novel

Across every window-forward system, “which column is the event-time clock” and “which input is the driving fact” are **declared by the user, never inferred**: Spark `withWatermark(col, delay) + join` direction; Flink `WATERMARK FOR ts + FOR SYSTEM_TIME AS OF`; Databricks `AUTO CDC SEQUENCE BY; dbt event_time=`; SQLMesh `time_column`; cube.dev `time_dimension`. All of them *trust* the declared column is monotone and that per-window evaluation equals whole-table evaluation; none proves it. The closest thing to an *inferred* version is Flink’s internal `RelModifiedMonotonicity` (optimizer-derived per-column update-increasing/ decreasing metadata) — but even that is seeded from a user-declared watermark.

smelt’s ambition to *derive* the event-time column from the SQL and *prove* it traces monotonically to a source partition column is stronger than anything shipped — best understood as “Flink’s watermark attribute, but inferred-and- proven from the projection rather than annotated at the source.” The theory (§12.2) bounds that ambition: the proof is necessarily incomplete (undecidable in general), so the honest design is *prove-where-you-can, declare-where-you-must* (§4.3) — reaching further toward inference than the annotate-only incumbents while keeping the declared escape hatch they rely on entirely.

Part 13 — Maintained refresh: the two axes, the contract, and hidden state

Parts 2–12 audit the window-forward camp. This and the next three parts work the **maintained** camp — the change-tracking side of §1.1 — by asking whether a single **maintained-relation** abstraction — cumulative generalized along two axes — can give smelt *both* camps’ trade-offs in one system: emulated on a plain engine like DuckDB, and **delegated to the engine’s native incremental-view maintenance (IVM) on a platform like Databricks**. These parts are positioning: they define the design space, land one load-bearing algebraic boundary (Part 14), recommend an ontology (Part 16), and leave the normative work to a future spec. The handoff from the batched camp is Part 11’s ordered slice: cumulative and self-referential batched models are exactly the shapes that read computed cross-window state — i.e. that keep maintenance state.

13.1 Axis I — state representation: *direct* vs *hidden*

Cumulative today is a single point. Two orthogonal choices open it into a space.

- **Direct state.** The value smelt stores *is* the value the user selects. Merging two partitions’ SUM gives a SUM; the stored column is the answer. This is today’s cumulative ([cumulative_aggregate.md](#) §“Aggregator allowlist”): the combiner’s output is directly presentable, so no indirection is needed.
- **Hidden (decomposed) state.** smelt stores an *intermediate* the combiner is closed over, plus a **presentation map** from that intermediate to the user-facing value. A mean is stored as (sum, count) and presented as sum / count. The user never selects sum or count; they select mean.

Hidden state is exactly the trick the delta engines use — Enzyme’s row-tracking, Dynamic Tables’ change streams, Materialize’s arrangements are all maintenance state the user’s SELECT never sees. The key move is that **smelt can keep that state itself**, in an ordinary table, and expose the user-facing value through a view:

```
state table  device_id, user_id, _sum_amount, _count_amount  ← smelt merges into this
presentation CREATE VIEW ... SELECT device_id, user_id,
    _sum_amount / _count_amount AS avg_amount FROM <state table>  ← user selects this
```

13.2 Axis II — maintainer: *smelt-driven* vs *engine-native IVM*

- **smelt-driven.** smelt emits the per-partition delta SELECT and the merge_into loop (today’s cumulative execution model, [cumulative_aggregate.md](#) §“Execution model”). For hidden state it additionally emits the presentation view. This works on **any** backend that has merge_into and views — DuckDB, plain Spark.
- **Engine-native IVM.** smelt emits a native maintained object — a Databricks materialized view / Enzyme-managed table, a Snowflake Dynamic Table — and lets the engine keep the hidden state *and* the presentation. smelt supplies the logical specification; the engine’s differential runtime does the maintenance. Only available on backends whose capability matrix advertises it. Note [models.md](#) already carries a **materialized_view** mode on the **storage** (materialization) axis, described as a “backend-managed persistent view” — the natural physical home for this maintainer.

The matrix:

	smelt-driven	engine-native IVM
direct state	cumulative <i>today</i> — SUM/COUNT/MIN/MAX/BOOL_*/BIT_*	native MV over an additive aggregate (redundant with smelt-driven, but free)

	smelt-driven	engine-native IVM
hidden, append-only (monoid)	AVG, variance, HLL-approx-distinct via (state table + view)	native MV; engine keeps the sketch
hidden, retraction (group)	reversible aggregates + delete/reprocess via a stored, invertible delta	the full delta camp: joins, DISTINCT, non-additive aggregates — anything the engine can maintain

Today’s **cumulative** is the top-left corner. The whole rest of the matrix is reachable, and — the key finding of Part 14 — the reachability boundary is **algebraic**, not backend-specific.

13.3 The unifying logical contract

All four corners uphold **one** contract, and it is cumulative’s contract generalized:

Maintained-relation equivalence. The *user-visible* value of the model equals what a full refresh would compute over the set of inputs processed so far.

Cumulative states this as cross-partition equivalence (`cumulative_aggregate.md` §“Cross-partition equivalence”): the end state after merging a *set* of source partitions equals a full refresh restricted to that set, independent of merge order. Generalizing changes only two words:

- “end state” → “**user-visible** value” — because with hidden state the stored columns are no longer the answer; the *view over them* is. The contract is asserted against what the user selects, not what smelt stores.
- The contract becomes **backend-uniform**. Whether smelt keeps (`sum`, `count`) in a DuckDB table or Databricks maintains the mean natively, the user sees the same logical relation with the same equivalence guarantee. Hidden state and the maintainer are *implementation detail beneath the contract*.

This reframes the whole design space as **one contract, four physical realizations** — the textbook logical/physical split smelt is built on (stated most crisply in `multi_backend.md` §Design; `architecture.md` gives the compiler pipeline that realizes it). It also means the contract does *not* mention monotone event-time: the maintained camp sidesteps the batched camp’s monotonicity price entirely (Part 15), because it tracks *what changed* rather than *assuming what is settled*. It is the end-state specialization of the §1.2 parent contract.

13.4 What hidden state collapses

Three entries currently sitting in cumulative’s §“Known Divergences / Open Questions” are **the same mechanism** — hidden state — seen three times:

Cumulative Known Divergence	What it needs	= hidden state?
AVG rewrite (“classifier refuses AVG; a future plan may rewrite to SUM/COUNT”)	store (<code>sum</code> , <code>count</code>), present <code>sum/count</code>	yes — decomposed monoid + presentation map
Reprocessing via delta history (“store per-partition deltas for reversible aggregators, enabling subtract-then-add”)	store enough to <i>invert</i> a partition’s contribution	yes — the stored delta <i>is</i> hidden group state
--auto staleness fidelity (“exactly the stale partitions ... needs the delta-history mechanism”)	the same per-partition delta history	yes — same store

Three deferred features, one enabling abstraction. That is strong evidence hidden state is a real organizing idea and not a speculative flourish: the cumulative spec already *reached for it three times* without naming it. Naming it once, as an axis, subsumes all three.

It also connects to the batched audit. **MIN/MAX** are supported append-only by Snowflake but require retraction state in Flink (§12.1 obs. 2); the monoid/group frame of Part 14 *names why* (they are a monoid but not a group). And the ordered / self-referential slice (§11.3) is exactly the maintained camp: cumulative and self-referential batched models are the two shapes that read computed cross-window state, i.e. that keep maintenance state.

Part 14 — The algebraic ladder: the maintained eligibility boundary

The reason to put algebra in a positioning doc: it draws the *exact* line between what each corner of the §13.2 matrix can express, with no hand-waving. Every combiner in play is an operation on stored state; its algebraic structure decides what is maintainable.

14.1 Monoid = append-only maintainable

A per-key aggregate is maintainable by merging partition deltas iff its combiner forms a **commutative monoid**: an associative, commutative binary operation $\oplus$ with an identity. Associativity + commutativity are precisely cumulative’s order-independence contract; identity is the empty partition.

- **Direct monoid** (stored value presentable as-is): **SUM** (+, 0), **COUNT** (+, 0), **MIN** (min, $+\infty$), **MAX** (max, $-\infty$), **BOOL_AND/OR**, **BIT_AND/OR/XOR**. This is today’s allowlist — and it is exactly the closed set of *directly presentable* commutative monoids over scalar columns. (The spec itself asserts only commutativity + associativity of the combiner; the identity element — the empty partition — is implicit there and made explicit here.)
- **Decomposed monoid** (needs a presentation map π): the *state* is a monoid element in a richer space; the user value is $\pi(\text{state})$.
 - **AVG** $\rightarrow$ state (sum, count) under componentwise +; $\pi = \text{sum}/\text{count}$.
 - variance / stddev $\rightarrow$ state (count, sum, sum_of_squares) (or a numerically stable Welford triple) under componentwise merge; $\pi =$ the closed form.
 - approximate **COUNT(DISTINCT)** $\rightarrow$ state = an HLL/sketch register vector under register-wise max; $\pi =$ the cardinality estimate. (Exact **COUNT(DISTINCT)** is *not* a bounded monoid — its state is the full set — which is why every delta engine treats exact distinct specially.)

The whole append-only half of the design space is “which commutative monoids can we store and present.” Direct is the subset where π is the identity. **Decomposed state is the entire content of the AVG/variance/approx-distinct unlock** — and it needs no engine support beyond a table and a view.

14.2 Group = retraction / delete / reprocess

Append-only monoids cannot *remove* a contribution. The moment inputs can change — late-arriving corrections, a reprocessed partition, a true source delete — the combiner must be **invertible**: a commutative **group** (a monoid with an inverse $\ominus$).

- **SUM**, **COUNT**, **BIT_XOR** are groups — $x \oplus y \ominus y = x$. These are precisely cumulative’s “reversible aggregators” ([cumulative_aggregate.md](#) §“Reprocessing semantics”). Subtract-then-add reprocessing works because they form a group.
- **MIN**, **MAX**, **BOOL_***, **BIT_AND/OR** are monoids **but not groups** — you cannot un-see a maximum without rescanning. This is the exact fault line §12.1 obs. 2 notices empirically: Snowflake

supports **MIN/MAX** *append-only* (monoid is enough to add) while Flink keeps *retraction state* for them (a non-group needs the raw multiset to handle a delete). Same fact, now named.

So the state-representation axis has **four** rungs — three that need no user opt-in, and one that does (§14.4):

direct monoid $\subset$ decomposed monoid (append-only) $\subset$ group (retraction) $\subset$ explicit multiset (opt-in)			
SUM, MIN...	+ AVG, variance, HLL	+ delete/reprocess for invertible ones	+ MEDIAN, exact distinct, MODE, quantiles (§14.4)

14.3 The boundary is where smelt-driven stops and native IVM begins

smelt-driven maintenance (a `merge_into` loop, optionally with a presentation view) can realize **any commutative monoid it can store, and retraction for the group subset**. That already covers AVG, variance, approximate distinct, and reversible reprocessing — a large, clean, *derivable-from-SQL* class, on DuckDB, with no engine IVM at all.

What it **cannot** self-maintain is the part of the delta camp that needs general retraction over arbitrary operators: incremental **joins** (bilinear — DBSP’s product rule), **DISTINCT** and exact **COUNT(DISTINCT)** (unbounded state), non-additive aggregates like **MEDIAN/PERCENTILE** (all-rows state). Those are maintainable, but only by a runtime that keeps per-operator differential state — which is what native IVM *is*. That is the honest boundary between the two maintainer columns: **smelt emulates the monoid/group aggregate slice; native IVM adds the general-operator slice smelt cannot keep state for**. That boundary is not fixed, though: for single-column holistic aggregates it *moves* once the user accepts unbounded state — the opt-in fourth rung of §14.4.

14.4 The opt-in fourth rung: explicit multiset state (bounded-domain Z-set)

The operators §14.3 hands to native IVM purely for *state-size* reasons — the holistic single-column aggregates **MEDIAN**, **PERCENTILE**, **MODE**, exact **COUNT(DISTINCT)**, and the **DISTINCT**-modified aggregates — are recoverable by smelt-driven maintenance if the user accepts state that is unbounded *in general*. This is a **space** trade, not a correctness one: the maintained-relation equivalence contract (§13.3) holds unconditionally for every operator below. What is unbounded is the number of rows in the state, never the fidelity of $\pi(\text{state})$.

The state is the **value-frequency multiset** — for each key group, the map $\text{value} \mapsto \text{count}$ over that column’s active domain. Merging partitions is componentwise count addition, so the multiset is the free commutative *monoid* over the domain; allow signed counts and it is the free abelian *group* — which is exactly the **Z-set retraction model** (Feldera/DBSP, §14.3 and References) restricted to a single column. “Store distinct values and counts” is therefore not an ad-hoc trick; it is opting one aggregate into a bounded-domain Z-set.

One state, many presentations. Because π is any pure function of the empirical distribution, a single multiset state serves:

- **MEDIAN**, any **PERCENTILE_CONT/DISC**, and arbitrary quantiles;
- **MODE**, entropy, gini — any functional of the distribution;
- exact **COUNT(DISTINCT)** (keys with count > 0) and the **DISTINCT**-modified aggregates **SUM(DISTINCT)** / **AVG(DISTINCT)** (needing only the key *set*, a cheaper sub-state than the full histogram);
- exact top-K / heavy hitters (the largest counts).

And because the signed version is a *group*, retraction is free for all of them — **including MIN/MAX**, the monoid-not-group cases §14.2 could add but not un-see. The full multiset is precisely the retraction state Flink keeps for MIN/MAX (§14.2; §12.1 obs. 2): keeping the whole distribution is what lets you delete the current maximum and recover the previous one.

Exact vs. approximate is the real axis. Most of these holistic aggregates also have a *bounded* decomposed-monoid realization needing no opt-in — the same relationship §14.1 draws between exact distinct and the HLL sketch:

aggregate	bounded, decomposed monoid (§14.1, no opt-in)	unbounded exact (opt-in multiset)
distinct count	HLL / sketch register vector	exact key set
quantiles / median	t-digest / KLL sketch	exact histogram
top-K / heavy hitters	Space-Saving / Misra-Gries	exact histogram
mode, entropy, DISTINCT-aggs	— (need the distribution)	exact histogram
ordered ARRAY_AGG/STRING_AGG	—	multiset of (sort_key, value); π re-sorts

So **MEDIAN** needs the opt-in rung only when *exact*; approximate **MEDIAN** is a t-digest decomposed monoid that belongs beside HLL in §14.1 — bounded state, no opt-in.

Why it must be opt-in and fail-loud. State size is $O(\text{active domain})$, unbounded for a high-cardinality column, so it cannot be the default — that would silently build state proportional to input size. The fitting posture is fail-loud + lower-don’t-reject ([multi_backend.md](#) §Design “Lower, don’t reject”): by default the classifier refuses an unbounded-state aggregate and suggests either the bounded approximate form or full-refresh; the user opts in by asserting the domain is bounded, and the runtime keeps a cap that falls back to full-refresh if the multiset exceeds it. This keeps derive-don’t-declare intact — the SQL still just says **MEDIAN**; the opt-in is a *space-budget assertion*, not a strategy knob that changes the contract.

The part that stays firmly native-IVM-only is the genuinely multi-relation delta camp: incremental **joins** (bilinear), and operators whose state is unbounded in a dimension the user cannot cap. The fourth rung moves the boundary for *single-column holistic aggregates*, not for the general-operator slice.

Part 15 — Emulation vs delegation

15.1 (state table + view) on DuckDB *is* what Enzyme does natively

The presentation-view mechanism is not a DuckDB workaround — it is a *portable reimplementaion of the engine trick*. Enzyme keeps hidden row-tracking state and serves a clean logical MV; smelt keeps hidden (sum, count) state and serves a clean logical view. Same logical object, two maintainers. This is why the same **refresh** declaration can compile to either without changing the user’s mental model or the equivalence contract (§13.3):

- **DuckDB / plain Spark** — smelt maintains: state table + `merge_into` loop + presentation view. Capability required: the `merge_into` primitive (`supports_merge`) + views (both already present).
- **Databricks** — the engine maintains: `CREATE MATERIALIZED VIEW ...` and Enzyme’s runtime keeps state + presentation. Capability required: `supports_native_ivm`.

15.2 Capability model

This slots into the existing `multi_backend.md` capability matrix, which already carries a “native materialized view” notion (§Semantics “Required lowerings”: “No backend today emits a native materialized view: DuckDB and both Spark profiles take the table fallback ... a real one would be a Databricks-only capability”). Two flags — named after the matrix’s existing `supports_*` convention — express the space:

- **`supports_native_ivm`** — the backend can maintain a declared query as a native incremental view. `true` $\rightarrow$ delegate; `false` $\rightarrow$ smelt-driven (`state table + view`) fallback. This is the standard `multi_backend.md` **lower-don’t-reject** posture (§Design “Lower, don’t reject”): a missing capability is a lowering obligation, not a user-facing error. (Part 17 adds one carve-out: the `materialized_view mode` is a declared commitment to engine-owned freshness, so for that mode a missing capability is a hard error, not a lowering — §17.7.)
- **`supports_retraction`** — whether the maintainer can invert contributions (delete / reprocess). smelt-driven sets this **true only for the group subset** (§14.2); native IVM sets it **true** generally. Drives whether a reprocess is accepted or refused-with-`--full-refresh` (cumulative’s current v1 policy).

The user-facing surface stays a single **refresh**: declaration; direct vs decomposed vs native is **derived** (SQL shape + capability), never declared — the derive-don’t-declare posture cumulative already takes for `unique_key` and aggregators (`cumulative_aggregate.md` §Design), extended here to state representation, and the one §11.4 argues for ordering. `AVG` in the SQL $\Rightarrow$ decomposed; `has_native_ivm` $\Rightarrow$ delegate; otherwise smelt-driven.

15.3 The two hazards emulation introduces

Delegation inherits the engine’s correctness; **emulation is smelt’s to get right**, and hidden state adds two concerns the direct case never had:

1. **Presentation-view consistency under partial merge.** Cumulative merges N partitions as N transactions and tolerates partial progress (`cumulative_aggregate.md` §“Execution model”). A view over the state table is then always well-defined *as a function of current state* — `sum/count` of a half-merged state is the mean of what has been merged. That is the same partial-progress semantics cumulative already documents, lifted through π . The requirement is only that π be a pure function of a single consistent snapshot of the state row — which rules out a π that reads other rows or other tables.
2. **Atomic state/view swap on schema change.** Adding a decomposed aggregator changes the state table’s shape *and* the view. The pair must move together, and a full rebuild of decomposed state requires rescanning source history — the same backfill limitation cumulative’s §“Known Divergences” (schema evolution) already names, now also touching the view definition. Emulation must treat (`state table, view`) as one atomically-swapped unit.

Part 16 — Ontology: the maintained umbrella

Recommendation: introduce a **maintained** refresh concept as the umbrella, and make **cumulative** its first named member — the `{direct, smelt-maintained, monoid}` instance. Do *not* stretch the word “cumulative” to cover the whole space.

The argument, in order of weight:

1. **The two axes are orthogonal to “it is an aggregate.”** State representation (direct/decomposed/group) and maintainer (smelt/native) are properties of *how state is kept*, not of *what the query computes*. `scd2`, `latest_value`, and `accumulating_snapshot` (20260522 §“Sibling rules”) are maintained relations that keep hidden state and defer/emulate identically, yet are not aggregates at all. An umbrella keyed on “maintained relation with hidden state and an equivalence contract” holds all of them; a generalized “cumulative” would have to mean “any maintained relation,” at which point the name actively misleads (a slowly-changing dimension is not cumulative anything).
2. **The contract generalizes cleanly; the *name* does not.** §13.3 shows the equivalence contract lifts verbatim to the whole space. The word “cumulative” describes *one* combiner behaviour (running accumulation), so using it as the umbrella severs the tight name↔contract fit smelt values elsewhere (`models.md`’s insistence that each axis value names one contract). Keeping `cumulative` = the additive-aggregate member preserves that fit.
3. **It matches the rule-composition posture already chosen.** 20260522 explicitly prefers “narrow, composable rules ... separate sibling rules per pattern ... compose better than one generic MERGE rule with enough knobs.” An umbrella-with-members *is* that posture; generalize-cumulative is the “one rule with enough knobs” it rejected.
4. **It leaves the DuckDB-emulation / native-IVM choice where it belongs — in physical execution, invisible to the surface.** Under the umbrella framing the maintainer axis is a `multi_backend` lowering decision (§15.2), not a new user-facing refresh value. Generalize-cumulative would tempt a `cumulative: { native: true }` knob — surfacing physical execution, exactly the metadata-vs-SQL drift the cumulative spec and rule-composition rationale both fought to avoid.

What this recommendation is *not*: it is not a proposal to rename or restructure `cumulative` now. Cumulative ships as-is. The umbrella is a *conceptual* home introduced when the first sibling (or the first hidden-state member) is specified; until then it is a documented direction. The concrete near-term surface implication is only that `models.md` §“Refresh axis” should describe `cumulative` as *one member of a maintained family*, leaving room for `maintained` / siblings, rather than as a one-off peer of `incremental`.

The runner-up (generalize `cumulative`) is worth stating so the rejection is legible: it has the smallest surface (no new concept) and would be right *if* the space were only “more aggregates.” It fails because the space is also non-aggregate maintained relations (the siblings) and a physical maintainer axis — two things the word cannot absorb without becoming a misnomer.

Part 17 — The user surface: explicit materialization modes

Parts 1 and 16 settled the *conceptual* ontology — a `maintained` umbrella, a `processed-input-equivalence` parent contract, and a stateless/stateful spine. This part settles the **user surface**: what a person actually writes in a model’s frontmatter or `smelt.yml`. It also fixes the naming of the window-forward mode.

17.1 The materialization mode is *declared*, not derived

The rest of this design — and the batched audit — leans hard on **derive-don’t-declare**: eligibility, lookback, monotonicity, and the algebraic rung are all read *from the SQL*, never restated in YAML

where they can drift. The materialization mode is the deliberate exception, for a concrete reason:

The mode is a physical commitment that is not cheaply reversible. Moving a model `full` $\rightarrow$ `batched` $\rightarrow$ `cumulative` $\rightarrow$ `native-IVM` rebuilds hidden state, changes what downstream may assume (a partitioned table vs a keyed lookup), and re-plumbs freshness ownership. It is a migration, not a recompile.

Choosing that silently for the user would be as wrong as silently repartitioning a table under them. So the division of labour is:

	Declared (user)	Derived (smelt)
what	the materialization mode (one word per model)	the algebraic rung (Part 14) the SQL lands on
why	not cheaply reversible; the user owns the physical commitment	mechanically true of the SQL; restating it invites drift
when wrong	smelt picks a mode $\rightarrow$ silent, costly migration	user restates algebra $\rightarrow$ drifts from the query

The derive-work does not disappear — §17.6 shows it changes *job*, from **chooser** to **validator**.

17.2 A flat enum of peers is *not* the dbt footgun

§1.3 rejected a selector-with-a-strategy-knob (`refresh: incremental + strategy: window | merge`) because the sub-knob silently swaps the equivalence contract under one name. A flat enum of **distinct named peers, each naming one contract**, is the *opposite* of that, and is exactly the “clearly-distinct children beneath the parent contract” §1.4 endorsed — extended here to the full family. Name $\leftrightarrow$ contract fit (`models.md` §“Refresh axis”) is intact: each value means one thing.

The distinction that keeps this honest is **declare-as-selector vs declare-as-assertion**:

- *declare-as-selector* (the footgun) — the declaration **changes what runs** under a shared name. `strategy: merge` silently changes invariants. Rejected.
- *declare-as-assertion* — the declaration **names a distinct contract** (or is checked against derived truth and errors on mismatch), and never silently varies invariants. The peer enum is this. So is the optional ceiling guardrail in §17.6.

17.3 The modes

```

---
refresh: full           # recompute everything each run
---
---
refresh: batched        # process new data in batches, forward along a monotone
partition_column: event_date # partition_column (a timestamp – or a monotone integer)
---
---
refresh: cumulative     # maintained running aggregate – keyed lookup, smelt owns freshness
---
---
refresh: versioned      # SCD Type 2 – keep every version of a key with a validity interval
---
---
refresh: latest_value    # SCD Type 1 – keep only the current row per key

```



```

---
---
refresh: materialized_view # maintained – engine owns freshness (native IVM); see §17.8
---

```

refresh:	correctness contract	output shape	freshness owner	hidden state	camp (§1.1)
full	trivial (recompute)	table	smelt (per run)	none	—
batched	per-partition slice	partitioned, partition_column	smelt (per run)	none	window-forward
cumulative	end-state	keyed lookup	smelt (per run)	O(keys)	maintained
versioned	end-state (interval-keyed)	key + validity interval	smelt (per run)	O(keys, open)	maintained
latest_value	end-state	keyed lookup	smelt (per run)	O(keys)	maintained
materialized_view	end-state	keyed lookup	engine (continuous)	engine-managed	maintained

Project-level default with per-model override (following the existing `smelt.yml` model-config cascade — per-directory default, model frontmatter wins):

```

# smelt.yml
models:
  refresh: full # project default
  marts:
    refresh: cumulative # everything under marts/ is maintained unless a model overrides

```

17.4 Naming rationale

batched (renaming the mode the rest of this document — and the shipping surface — calls `incremental`). Three names were considered and rejected before landing here:

- **incremental** is *overloaded*. `cumulative`, `versioned`, and `materialized_view` are all “incremental” in the broad, incremental-view-maintenance sense — the §1.2 terminology collision. Retiring `incremental` as a *value* frees it to be the **family word** in prose (“`batched`, `cumulative`, and `materialized_view` are all incremental approaches — here is how they differ”), which resolves the collision instead of inheriting it.
- **partitioned** fails to distinguish: *every* mode’s table can be physically partitioned (a `full` or `cumulative` table just as much). It names a storage property the modes share, not what makes this one different.
- **time_partitioned** re-inherits that storage conflation *and* adds a lie — the partition key need not be time (see below), so a monotone-integer model would be confusingly “time”-partitioned.
- **batched** names the axis that actually distinguishes the mode. `batched` and `full` retain the *same contents* (a plain complete table) and differ only in **build method** — recompute-everything vs process-the-new-tail-in-batches. So `full` ↔ `batched` is both legible and structurally true. Accepted wart: `batched` is conceptually adjacent to dbt’s `microbatch`; the word is distinct and the trade was taken for legibility.

The **partition key must be monotone** (a timestamp *or* an ever-increasing integer — sequence id, offset, watermark). That “clock-like” requirement — what licenses “earlier partitions are settled” — lives as a property of the **key** (`partition_column`, validated against the monotonicity primitive, Part 4), *not* as a word baked into the enum. This is why the mode name deliberately says nothing about time.

versioned / **latest_value** (the SCD2 / SCD1 patterns, named without the vendor “SCD” jargon). The pair is deliberately symmetric — **latest_value** (overwrite, keep current) reads against **versioned** (keep every version with validity intervals) as exactly the SCD1↔SCD2 contrast, without either name mentioning “slowly-changing dimension.” Both are maintained siblings of **cumulative** (Part 16; 20260522 §“Sibling rules”).

17.5 Freshness owner distinguishes **cumulative** from **materialized_view**

cumulative and **materialized_view** share the *correctness* contract (end-state equivalence, §13.3) — §15.1 shows (state table + view) on DuckDB is native IVM by another maintainer. What earns them **peer names** is a different *operational* contract — **who owns freshness**:

	cumulative	materialized_view
freshness model	pull — correct as of the last smelt build	push — engine keeps it current continuously
cadence owner	smelt	the engine
“is it up to date?”	after the last run	between runs too

That is a genuinely different commitment (a different answer to “is this table fresh, and who is responsible”), which is why it is a peer value rather than a hidden physical detail.

17.6 The derive-work becomes a *validator*, not a chooser

Because the mode is declared, the algebraic ladder (Part 14) stops *choosing* and starts *validating the declared mode*:

A model with **refresh: cumulative** over a **MEDIAN: smelt** derives that **MEDIAN** is holistic → not monoid-maintainable → **emits a diagnostic** (“**MEDIAN** is not maintainable at the additive rung; declare a **bounded_domain** to maintain it exactly (§14.4), or use **refresh: full**”). It does **not** silently downgrade to **full** or switch modes.

So the rung is **output, not input** — surfaced in **explain/plan** and diagnostics (“maintains at the retraction rung, keeping O(keys) state”), never hand-picked. The eligibility boundary is:

rung = f(source mutation profile , aggregate algebra) — the algebra half is derived from the SQL; the **source mutation profile** (append-only vs mutable/restatable) is the one world-fact smelt *cannot* derive (it cannot know if an upstream table is updated in place), so it is the honest thing to *declare* — on the source, shared by every consumer. The §14.4 bounded-domain opt-in is the same category of world-fact.

Users may additionally assert a **ceiling guardrail** (declare-as-assertion, §17.2): “error if this model cannot be maintained append-only / without a full refresh.” It never changes execution; it pins cost intent and fails loudly on drift.

17.7 **materialized_view** has no silent fallback

Because the modes are **peers** and smelt does not choose for the user (§17.1), **materialized_view** cannot silently degrade:

1. **Engine has no native IVM** (e.g. DuckDB) → **refresh: materialized_view** is a **hard error**: “*materialized_view requires native IVM; this engine has none — use cumulative for smelt-driven maintenance.*” Smelt does not quietly substitute **cumulative** — that would swap the declared mode.

2. **Engine has IVM but rejects the query** (Enzyme’s `MATERIALIZED_VIEW_NOT_INCREMENTALIZABLE`, Part 18) → also a hard error, carrying the engine’s reason.

`cumulative` and `materialized_view` gate on the *same* algebraic ladder; `materialized_view` simply carries the **extra** eligibility constraint of engine-incrementalizability. The inability to rescue the user into a different mode is the honest price of peer status — and the reason the enum stays coherent.

17.8 `materialized_view` stays a peer (decision)

`versioned` *looked* like it exposed a seam — the maintained family seemed to have two independent axes (*pattern*: `cumulative` / `versioned` / `latest_value`; *freshness owner*: `smelt-pull` vs `engine-push`) that a peer enum cannot factor, threatening a combinatorial blow-up (`versioned-pull`, `versioned-push`, ...). It dissolves once the **asymmetry between the two maintainers** is made explicit:

- **smelt-driven maintenance requires a *named pattern*.** `smelt` can only maintain a relation itself if it owns the combiner — hence `cumulative`, `versioned`, `latest_value` are specific, recognised patterns.
- **engine-driven maintenance requires *no* named pattern.** `materialized_view` hands the SQL to the engine’s native IVM, which incrementalises arbitrary eligible SQL. To get an engine-maintained SCD2 you **write the SCD2 logic in SQL and declare `refresh: materialized_view`** — there is no `versioned` + `native` cell to fill, because native maintenance never needed the pattern name.

So the axes do not multiply, and no `maintained_by`: modifier is required. **Decision: keep `materialized_view` as a peer.** `versioned` / `latest_value` are the *smelt-maintained* SCD patterns (`smelt` owns the combiner); their engine-maintained counterpart is not a variant of them but the generic `materialized_view` over hand-written SQL.

A future modifier is a *different* concern. If a single engine exposes *multiple native IVM implementations* of the same view (distinct refresh algorithms / incremental strategies), `smelt` would pick a sensible **per-engine default and let the user override it** — a physical-strategy override scoped *inside* `materialized_view`, engine-specific and defaulted. That is not a logical-mode selector, so it does not reintroduce the strategy footgun (§1.3, §17.2). Deferred until an engine actually presents the choice.

17.9 Recommendation

- **Adopt the explicit-mode surface.** The materialization mode is user-declared and stable; `smelt` never chooses it. The value enum: `full` | `batched` | `cumulative` | `versioned` | `latest_value` | `materialized_view`.
- **Rename the window-forward mode `batched`** (retiring `incremental` as a value, keeping it as the family word); require `partition_column` to be monotone.
- **Name the SCD patterns `versioned` (Type 2) and `latest_value` (Type 1).**
- **The algebra validates, never chooses** — a mode the SQL cannot satisfy is a diagnostic, and `materialized_view` never silently falls back.
- **`models.md` §“Refresh axis” phrasing.** Present the values as peers each naming one contract, grouped by the stateless (`batched`) / maintained (`cumulative`, `versioned`, `latest_value`, `materialized_view`) spine — not as a flat enum, and not under a strategy sub-knob.
- **Keep `materialized_view` a peer (§17.8).** Engine-maintained non-aggregate shapes are expressed as SQL under `materialized_view`, not as `pattern` + `native`; defer any physical-strategy modifier until an engine exposes multiple native IVM implementations.

Part 18 — Open questions

Combining the two camps and the surface into one document changes the open-question ledger in both directions: several questions that were open when the camps were analysed separately are settled by the combination (§18.1), the genuinely open ones are collected in §18.2, and the combination itself creates a few new ones (§18.3).

18.1 Settled by the combined scope

- **The batched ↔ maintained routing handoff — resolved: reject-and-suggest, never auto-route.** The batched audit left open whether a cross-partition `UNBOUNDED PRECEDING` running total (a maintained relation written in batched clothing, §8.3) should be *routed* to the maintained camp automatically or rejected with a suggestion — “the two workstreams need one agreed line.” The declared-mode surface supplies the line: the mode is a user-owned physical commitment smelt never chooses (§17.1), so the window-cluster classifier’s verdict becomes a **diagnostic** — “this reach is cumulative; use `refresh: cumulative`” — and never a silent mode switch (§17.6). One agreed answer for both camps.
- **materialized_view fallback — resolved: hard error.** The maintained camp’s capability question — when the engine lacks IVM or rejects the query, does smelt fall back to smelt-driven maintenance or to full refresh? — is answered by mode peer-ness: neither. A declared `materialized_view` that the engine cannot maintain is a hard error naming the reason (§17.7), because a silent substitution would swap the declared freshness contract.
- **Where the decomposability classification lives — resolved: Part 14.** The batched audit established that holistic aggregates need no batched gate and that the decomposability whitelist “belongs in the cumulative spec” (§9.3); the maintained camp independently needed exactly that classification as its eligibility ladder. In the combined frame it is one classification (Part 14) with two consumers: the batched validator (where it imposes nothing) and the maintained validator (where it is the rung).
- **The “incremental” terminology collision — resolved by the batched rename.** smelt’s window-forward `incremental` and the industry’s “incremental view maintenance” name opposite camps (§1.2). Retiring `incremental` as an enum value and keeping it as the family word (§17.4) resolves the collision instead of documenting around it.
- **Validation methodology (property tests vs. curated fixtures) — resolved by the primitive build.** The generative soundness oracle is the property-test form and the reusable asset; the hand-written harnesses are retained as deterministic seed cases (§4.8).

18.2 Still open

Batched camp — construct-level:

- **Aggregating-branch unions** (Strategy B, §5.4): worth it, or steer users to a CTE that unions raw events then aggregates once at the outer select (which Strategy A already handles)?
- **Static-seed branches** (§5.5 case 2, constant event-time): reject, or model as a once-computed contribution to a single partition?
- **UNION/INTERSECT/EXCEPT:** the algebra distributes (§5.2/§5.3), but demand is unclear. Gate on a real use-case before building.
- **Subquery body classification (B4/E2):** can “transparent” (project/filter/ rename only) be reliably distinguished from aggregating / order-sensitive bodies by static analysis of the subquery `SELECT`, or does the safe slice need a whitelist of recognised shapes? (§6.2)
- **CTE parity (B4/E2):** the derived-table and CTE spellings are the same query (§6.3) — should

the fix unify them by classifying CTE bodies, and does closing the current CTE bypass risk newly-rejecting queries that build today?

- **Group-aligned aggregating subqueries:** an aggregation whose `GROUP BY` key $\supseteq$ `partition_column` is window-local and safe (§6.2 row 3) — is it worth supporting directly, or steer users to the flat aggregate the outer select can already express?
- **Join hazard as a design constraint (Part 7):** the timeseries-dimension-as-lookup misfilter (§7.2, J3, 400 violating rows) is not treated as a live incident to patch (smelt is early-stage) but as a **constraint the eligibility model must satisfy**: whatever gate lands must window only the driving fact, so J3 goes to 0 by construction. The open design choice is *how* the driving fact is identified — inferred, or declared (see next question).
- **Reuse of declared joins: cardinality (Part 7):** the planner already trusts declared cardinality for join elimination (§20E caveat). Should incremental eligibility reuse that same declaration to license fact-only pushdown, and does leaning on an unverified declaration for *correctness* (not just optimisation) raise the stakes of the §20E soundness caveat?
- **FOLLOWING frames / forward reach (§8.3):** a bounded `RANGE ... INTERVAL 'a' FOLLOWING` frame has a derivable forward reach in principle, but the settledness problem is new — window `W` differs from a later full refresh until the source is complete through `hi + a`. Watermark- style delay, or tail-rewrite (with its §3.2 composition trap)? Also `source_bounds` Form A currently parses `PRECEDING` frames only.
- **Scalar subqueries over bounded sources (§2.2 addendum):** gate them with an E2-style rejection that names the construct, or teach `inject_source_filters` to leave refs inside scalar subqueries un-windowed (they are window-invariant lookups by construction, like the §7.4 non-driving join inputs)?
- **GROUPING SETS/ROLLUP/CUBE (§2.2 addendum):** reject super-aggregate grouping outright for batched models, or admit exactly the grouping sets in which *every* set contains `partition_column` (the others produce the `NULL`-partition cross-window rows)?
- **Membership/grouping non-determinism (§9.2):** the payload opt-in admits non-determinism confined to stored output values. A non-deterministic *predicate* or *grouping key* (`WHERE RANDOM() < 0.5`, `GROUP BY` on a random bucket) changes which rows exist or how they aggregate — which two full refreshes would also vary, so the sharpened contract (clause 2) would *permit* it. But an incremental build freezes each window’s membership at run time, and reconciling that against an all-at-once full refresh is a harder object than a payload value. Admit it (the contract allows it), or keep it rejected as out-of-envelope for the `DELETE+INSERT` mechanism? Needs its own argument before the opt-in is widened past projections.

Batched camp — mechanism-level:

- **Classifier returns a pushdown depth, not a boolean (Part 3):** how much of the “deepest safe injection point” walk can reuse the existing `source_bounds/temporal` analysis versus needing a new operator-by-operator pass? Is a per-source injection point always resolvable statically, or are there shapes where we must fall back to the outer clamp?
- **Retiring the outer clamp when there is no lookback (§3.3/§3.5):** is it safe to drop the outer `inject_time_filter` for the transparent slice, or should the outer clamp stay as a cheap correctness backstop even when redundant with a source filter on the same window?
- **One bound derivation instead of two (§3.5):** unifying the output-clamp window and the per-source bound (`execute.rs:895` vs `:913`) into a single per-source derivation — worth doing as part of this work, or a follow-on refactor once the classifier lands?
- **Migrating to exact output clamps (§3.2 / §8.2 / §11.2):** today’s runtime widens both the clamp *and* the `DELETE` by the derived lookback, which re-writes margin rows from clipped scans (the confirmed §3.2 under-read) and makes adjacent runs’ writes overlap (§11.2). Is the

exact-clamp design adopted wholesale — and what then carries the late-data use case, whose *point* is to re-write earlier partitions (§8.6 axis (b))?

- **Run↔partition granularity check (Part 10):** the $g_run \geq g_part$ invariant (§10.2) is unchecked today. Should it be a hard validation, or should smelt *auto-coarsen* the run window to the partition granularity when a finer cadence is configured? And is deriving g_part from the partition-column transform unit (via the Part 4 primitive) sufficient, or are there partition columns whose granularity is not statically legible?
- **Self-referential batched models (Part 11):** are models that read their own prior partitions (smelt.ref to self) in scope at all, or a non-goal that should be steered to **refresh: cumulative**? The combined frame sharpens the stakes — the §1.4 spine names the shape stateful-ordered while it still executes as partition DELETE+INSERT. If in scope, the *ordered* execution property (§11.4) must be derived and enforced (no parallel backfill); if a non-goal, the self-edge should be a named rejection rather than a silently mis-parallelised build.

Maintained camp:

- **How much decomposition is derivable vs. needs a registry?** AVG $\rightarrow$ (sum,count) and variance are mechanical rewrites. Approximate distinct needs a *chosen* sketch (HLL precision). Is the decomposed-monoid set a closed, hard-coded rewrite table (like the current combiner lookup), or an extensible registry? The closed-table answer matches cumulative’s “fixed allowlist, not a registry” stance; revisit only when a concrete sketch motivator appears.
- **What is the opt-in surface for unbounded-exact state (§14.4)?** Exact MEDIAN/MODE/quantiles/DISTINCT-aggregates are maintainable via the value-multiset at 0(active-domain) state. How does the user assert the domain is bounded — a per-model annotation, a domain-size hint, or a runtime cap with full-refresh fallback? And is the exact-vs-approximate choice (multiset vs. t-digest/HLL) derived from a fidelity request or declared? The SQL (MEDIAN) should stay the operator source of truth; the opt-in should be a space-budget assertion, not a contract-changing strategy knob.
- **Presentation-view purity.** §15.3 requires π to be a pure function of a single state row. Is that guaranteed by construction from the decomposition rewrite, or does it need a classifier check (reject a π that references another table / window / row)?
- **Retraction without a monotone event-time.** The maintained camp does *not* need monotone event-time (§13.3). But smelt-driven retraction still needs to know *which* prior contribution to invert — i.e. it needs change-tracking on the source (a delta history / side table). What is the minimum source-side machinery smelt must keep for group-state retraction on DuckDB, and is it worth it vs. simply delegating retraction to native IVM only?
- **Native-IVM delegation scope.** Which engines expose a usable IVM surface (Databricks Enzyme; Snowflake Dynamic Tables; anything else)? The fallback question is settled (§18.1: hard error for the `materialized_view` mode), but the conformance work — capability detection, surfacing the engine’s own eligibility errors legibly — remains a `multi_backend` question.
- **Downstream pushdown is unchanged (a boundary to state, not really open).** A maintained relation has a unique key and no partition column, so downstream consumers treat it as a lookup exactly as cumulative output is treated today (`cumulative_aggregate.md` §“Output shape”). Hidden state does not change this — the *view* is the lookup; the state table is never a dependency target. Worth stating explicitly in any future spec so nobody tries to push a filter into the state table.
- **Does the umbrella subsume the sibling rules, or sit beside them?** Part 16 places `scd2/latest_value/accumulating_snapshot` in the same space, but they were sketched as *separate rules*. Is maintained an abstract contract the sibling rules each *implement* (shared execution, per-rule classifier), or a peer refresh value? Settling this is the first job of the umbrella’s own spec.

18.3 Newly created by the combined scope

- **Monotone-integer partition keys (§17.4 vs. Parts 4, 8, 10).** The `batched` surface deliberately admits a non-temporal monotone key (sequence id, offset, watermark), but the entire `batched` analysis machinery is time-typed: the monotonicity whitelist is built from temporal transforms (`DATE_TRUNC`, `INTERVAL` offsets, §4.2), lookback margins are `INTERVALs` (Part 8), `Offset::Seconds` folds temporal shifts (§4.5), and granularity alignment compares calendar units (Part 10). What are the integer analogs — integer offsets and bands are easy, but what is `g_part` for an offset key, and what does `timeseries.granularity` mean? The surface decision creates a concrete generalization work-item the audit never needed.
- **Mode migration paths (§17.1).** Declaring the mode a “migration, not a recompile” names the cost but provides no mechanism. What happens when a model’s declared `refresh:` changes — does `smelt build` detect the mismatch against existing physical state (a partitioned table where a keyed lookup is now declared), refuse until `--full-refresh`, or offer a migration? The per-camp docs never had to answer this because neither owned the enum.
- **Unifying the gate surface with the mode validator (§2.2 vs. §17.6).** The `batched` camp’s five enforcement pathways, `safety_overrides.allow_*` escape hatches, and `--allow-downgrade` all predate the declared-mode surface. Under `declare-as-assertion` (§17.2), do the B-group overrides survive as-is, become per-mode validator assertions, or get subsumed by the ceiling guardrail? And the silent-downgrade behaviour of `--allow-downgrade` (build as full refresh) sits awkwardly beside §17.6’s “never silently switch modes” — the two postures need reconciling. Relatedly, diagnostics and config keys say `incremental` today; the `batched` rename has a migration cost the rename decision did not price.
- **One orchestrator signal across camps (Part 11 vs. §17.5).** Part 11 derives ordered-vs-window-independent for `batched` models; maintained modes are inherently ordered; `materialized_view` removes `smelt` from the freshness loop entirely. Does the orchestrator consume one unified execution-planning signal (parallelisable / ordered / engine-owned), and how do `--auto` staleness and DAG scheduling treat a model whose freshness `smelt` does not own?

Non-goals

- Broadcast/dimension `UNION` branches that must appear in every partition (§5.5 case 3) — that is a `JOIN`, not a set operation.

References

External prior art cited in Parts 4, 12, and 13–16. Grouped by theme; every peer-reviewed entry was confirmed against at least one authoritative index (DOI, arXiv, dblp, or official venue). Two items are flagged as non-canonical where noted.

Theory — incremental maintenance, monotonicity, and limits

- **Monotone = incrementally-maintainable (CALM).** Hellerstein, “The Declarative Imperative,” *SIGMOD Record* 39(1), 2010 — [doi:10.1145/1860702.1860704](https://doi.org/10.1145/1860702.1860704) (the CALM conjecture). Ameloot, Neven & Van den Bussche, “Relational Transducers for Declarative Networking,” PODS 2011 / *JACM* 60(2), 2013 — [arXiv:1012.2858](https://arxiv.org/abs/1012.2858) (the proof: monotone queries are *exactly*

- the coordination-free class). Hellerstein & Alvaro, “Keeping CALM,” *CACM* 63(9), 2020 — [doi:10.1145/3369736](https://doi.org/10.1145/3369736) (accessible synthesis). Monotone/non-monotone operator boundary: Abiteboul, Hull & Vianu, *Foundations of Databases*, 1995 — webdam.inria.fr/Alice.
- **Incremental view maintenance (classic).** Blakeley, Larson & Tompa, “Efficiently Updating Materialized Views,” SIGMOD 1986 — [doi:10.1145/16894.16861](https://doi.org/10.1145/16894.16861) (irrelevant updates). Gupta, Mumick & Subrahmanian, “Maintaining Views Incrementally,” SIGMOD 1993 — [doi:10.1145/170035.170066](https://doi.org/10.1145/170035.170066) (counting algorithm). Gupta & Mumick, “Maintenance of Materialized Views: Problems, Techniques, and Applications,” *IEEE Data Eng. Bull.* 18(2), 1995. Quass, Gupta, Mumick & Widom, “Making Views Self-Maintainable for Data Warehousing,” PDIS 1996 (self-maintainability via key/RI constraints — the provenance argument behind tracing `event_time` to a source key).
 - **DBSP / differential dataflow (modern, constructive).** Budiu, McSherry, Ryzhyk & Tannen, “DBSP: Automatic Incremental View Maintenance for Rich Query Languages,” *PVLDB* 16(7), 2023 (Best Paper) — [doi:10.14778/3587136.3587137](https://doi.org/10.14778/3587136.3587137) / [arXiv:2203.16684](https://arxiv.org/abs/2203.16684) (linear vs bilinear operator taxonomy = the batched safe slice; Z-sets = the §14.4 retraction model). McSherry, Murray, Isaacs & Isard, “Differential Dataflow,” CIDR 2013. Murray et al., “Naiad,” SOSP 2013 — [doi:10.1145/2517349.2522738](https://doi.org/10.1145/2517349.2522738).
 - **Streaming / watermarks.** Akidau et al., “The Dataflow Model,” *PVLDB* 8(12), 2015 — [doi:10.14778/2824032.2824076](https://doi.org/10.14778/2824032.2824076) (watermark = monotone completeness bound). Akidau et al., “MillWheel,” *PVLDB* 6(11), 2013. Begoli et al., “Watermarks in Stream Processing Systems,” *PVLDB* 14(12), 2021 — [doi:10.14778/3476311.3476389](https://doi.org/10.14778/3476311.3476389).
 - **Decidability limits.** Chandra & Merlin, “Optimal Implementation of Conjunctive Queries,” STOC 1977 — [doi:10.1145/800105.803397](https://doi.org/10.1145/800105.803397) (CQ equivalence decidable, NP-complete). Trakhtenbrot’s theorem (equivalence of full relational algebra/SQL undecidable) — Libkin, *Elements of Finite Model Theory*, 2004. Rice, “Classes of Recursively Enumerable Sets...,” *Trans. AMS* 74(2), 1953 — [doi:10.1090/S0002-9947-1953-0053041-6](https://doi.org/10.1090/S0002-9947-1953-0053041-6). Richardson, “Some Undecidable Problems Involving Elementary Functions,” *JSL* 33(4), 1968 — [doi:10.2307/2271358](https://doi.org/10.2307/2271358) (monotonicity of elementary expressions undecidable $\rightarrow$ whitelist is mandatory). Wang, “The Undecidability of the Existence of Zeros of Real Elementary Functions,” *JACM* 21(4), 1974 — [doi:10.1145/321850.321856](https://doi.org/10.1145/321850.321856).

Predicate pushdown & monotone-expression detection

- **Pushdown laws.** Garcia-Molina, Ullman & Widom, *Database Systems: The Complete Book* (2e), 2009, Ch. 16 §16.2 (the canonical σ -commutation law set). Selinger et al., “Access Path Selection...” (System R), SIGMOD 1979 — [doi:10.1145/582095.582099](https://doi.org/10.1145/582095.582099). Hellerstein & Stonebraker, “Predicate Migration,” SIGMOD 1993 — [doi:10.1145/170036.170078](https://doi.org/10.1145/170036.170078). Empty-grouping-set caveat corroborated by [PrestoDB PR #11297](#).
- **Monotone-expression detection (the closest production analogs of the Part 4 primitive).** ClickHouse `IFunctionBase::getMonotonicityForRange` + the `Monotonicity` verdict struct — [src/Functions/IFunction.h](#); mechanism + whitelist in [Altinity](#), “[Learning to appreciate monotonic functions in ClickHouse](#)”. Apache Iceberg partition transforms (`preserves_order` + `project(predicate)`) — [table spec](#), [PyIceberg transforms.py](#). Delta Lake generated columns (order-preserving generation-expression whitelist). Range-rewrite framing: Bolenok, “[Sargability of monotonic functions](#)”.
- **Negative baseline (won’t reason about monotonicity).** Oracle VLDB & Partitioning Guide §3.1; PostgreSQL §5.12 Table Partitioning; DuckDB zonemaps/statistics propagation.

Industry incremental/materialized-view engines (published eligibility rules)

- **Databricks** — [Enzyme incremental refresh](#); error class `MATERIALIZED_VIEW_NOT_INCREMENTALIZABLE`; [AUTO CDC / APPLY CHANGES](#). Spark Structured Streaming watermarks & joins.
- **Snowflake Dynamic Tables** — [supported queries](#), [refresh modes](#).
- **BigQuery materialized views** — [create](#), [intro](#).
- **Apache Flink** — [changelog processing](#); internal `FlinkRelMdModifiedMonotonicity` (FLINK-34702).
- **Materialize** — [temporal filters](#), [now/mz_now](#).
- **Feldera/DBSP** — [SQL docs](#) (DBSP paper above).
- **ClickHouse MVs** — [incremental materialized view](#).
- **dbt** — [incremental overview](#), [microbatch](#). **SQLMesh** — [model kinds](#), [incremental by time](#). [cube.dev](#) — [pre-aggregations](#).
- **Window-function eligibility & derived-vs-declared lookback (Part 8)**. Spark Structured Streaming watermarking & state eviction — [Databricks](#), “[Feature Deep Dive: Watermarking in Structured Streaming](#)”, [Spark Structured Streaming Programming Guide §window operations & watermarking](#). Flink `OVER`-window append-only vs Top-N updating & the rowtime-`ORDER BY` requirement — [Confluent](#), “[Changelog processing in Flink SQL](#)”. Snowflake Dynamic Tables window functions as blocking operators — [Optimize queries for incremental refresh](#), [supported queries](#). Databricks Enzyme `WINDOW_WITHOUT_PARTITION_BY` — `MATERIALIZED_VIEW_NOT_INCREMENTALIZABLE` error class. Declared source-lateness knobs (axis (b) of §8.6): dbt microbatch [lookback / microbatch strategy](#); SQLMesh [incremental-by-time interval tracking](#). ClickHouse [window view](#) (watermark-driven tumbling/hopping).

Non-canonical (illustrative only): Fink, “On Monotonicity in Relational Databases...,” Palantir Engineering, 2018 (blog); US Patent 8,176,035 (monotonicity tracking — existence evidence, not a proof). The scalar order-preservation lemma ($a \leq x < b \implies f(a) \leq f(x) < f(b)$ for monotone f) has no single canonical paper; it is standard order theory realised by the zone-map/micro-partition pruning systems above.